

5.1.5 答案与解析

一、单项选择题

01. D

树是一种分层结构，它特别适合组织那些具有分支层次关系的数据。

02. A

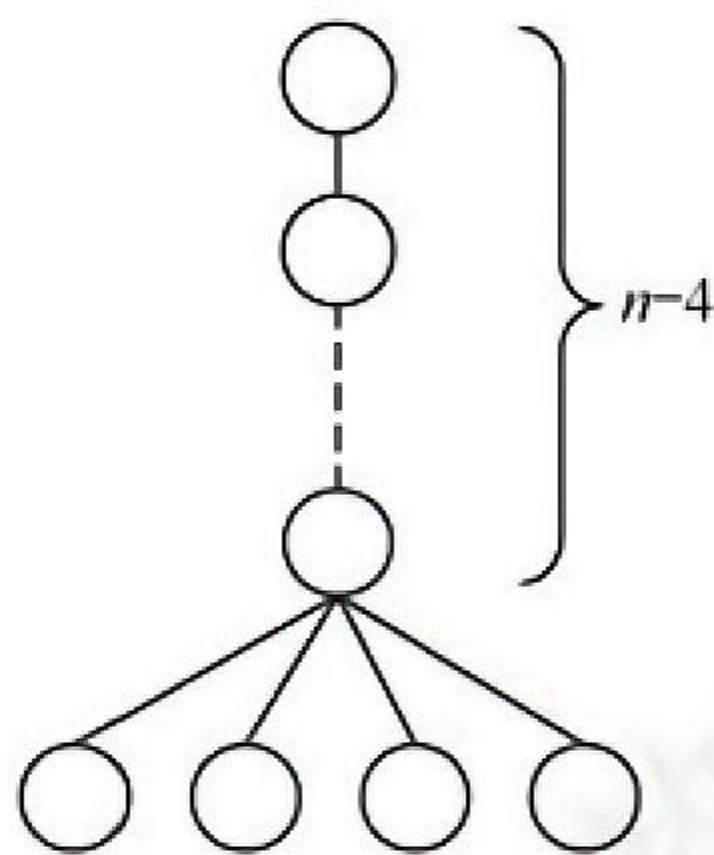
除根结点外,其他每个结点都是某个结点的孩子,因此树中所有结点的度数加 1 等于结点数,也即所有结点的度数之和等于总结点数减 1。这是一个重要的结论,做题时经常用到。

03. A

树的路径长度是指树根到每个结点的路径长的总和,根到每个结点的路径长度的最大值应是树的高度减 1。注意与哈夫曼树的带权路径长度相区别。

04. A

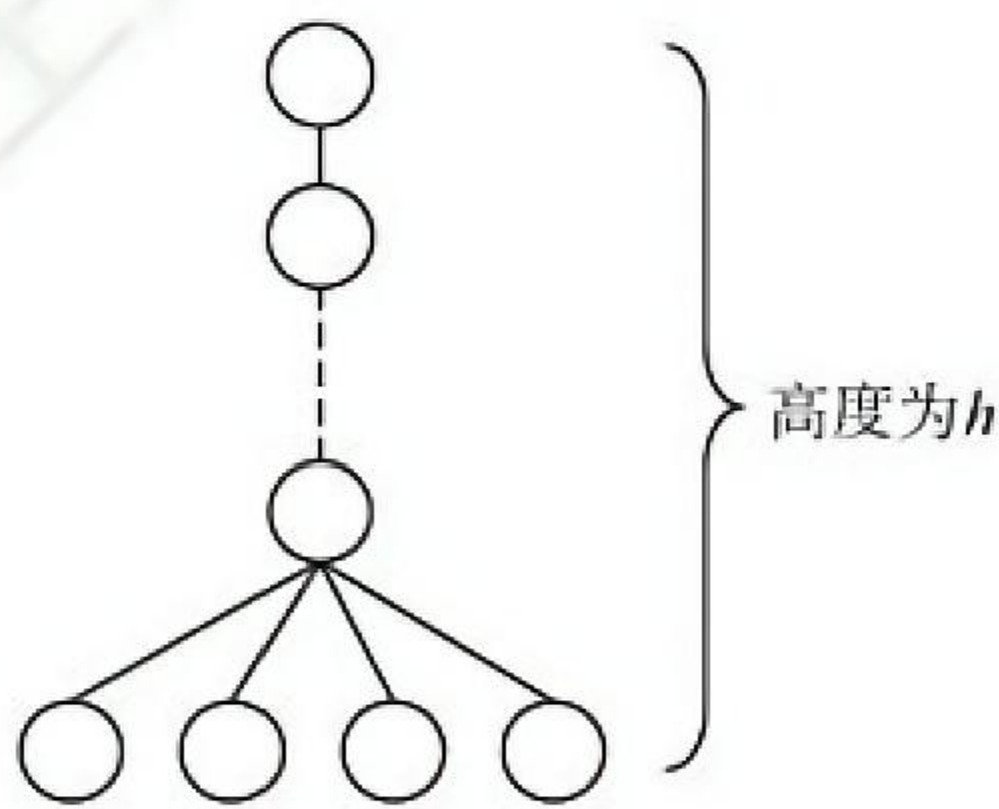
要使得具有 n 个结点、度为 4 的树的高度最大,就要使得每层的结点数尽可能少,类似下图所示的树,除最后一层外,每层的结点数是 1,最终该树的高度为 $n-3$ 。树的度为 4 只能说明存在某结点正好(也最多)有 4 个孩子结点, D 错误。

**05. A**

要使得度为 4、高度为 h 的树的总结点数最少,需要满足以下两个条件:

- ① 至少有一个结点有 4 个分支。
- ② 每层的结点数目尽可能少。

情况类似下图所示的树,结点个数为 $h+3$ 。



要使得度为 4、高度为 h 的树的总结点数最多,应使每个非叶结点的度均为 4,即为满树,总结点个数最多为 $1 + 4 + 4^2 + \dots + 4^{h-1}$ 。

对于上面的两题,应画出草图来求解,就能一目了然。

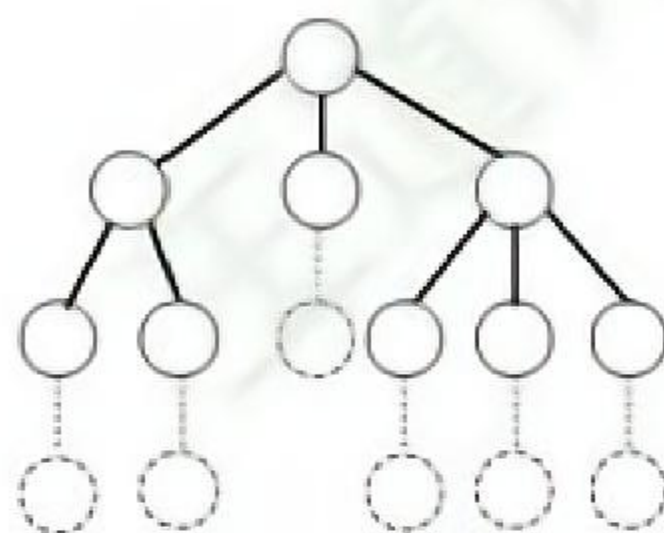
06. C

要求满足条件的树,那么该树是一棵完全三叉树。在度为 3 的完全三叉树中,第 1 层有 1 个结点,第 2 层有 $3^1 = 3$ 个结点,第 3 层有 $3^2 = 9$ 个结点,第 4 层有 $3^3 = 27$ 个结点,因此结点数之和为 $1 + 3 + 9 + 27 = 40$,第 5 层的结点数 $= 50 - 40 = 10$ 个,因此最小高度为 5。

07. B

总结点数 $n = n_0 + n_1 + n_2 + n_3 = 6 + n_1 + 1 + 2 = n_1 + 9$,总度数 $= n - 1 = n_1 + 2n_2 + 3n_3 = n_1 + 2 + 6 = n_1 + 8$,根据题目条件无法得出 n_1 的具体值,只能证明 n_1 是一个大于或等于 9 的任意整数。画出

满足题目条件的树，可以是如下图所示的一棵树，该树中无法确定 n_1 的具体数量。



08. D

设叶结点数为 N_0 ，总结点数为 N ，则 $N = N_1 + 2N_2 + 3N_3 + \cdots + mN_m + 1$ ，又因为 $N = N_0 + N_1 + N_2 + N_3 + \cdots + N_m$ ，所以 $N_0 = N_2 + 2N_3 + \cdots + (m-1)N_m + 1 = \sum_{i=2}^m (i-1)N_i + 1$ 。

09. B

设树中度为 i ($i = 0, 1, 2, 3, 4$) 的结点数分别为 n_i ，树中结点总数为 n ，则 $n = \text{分支数} + 1$ ，而分支数又等于树中各结点的度之和，即 $n = 1 + n_1 + 2n_2 + 3n_3 + 4n_4 = n_0 + n_1 + n_2 + n_3 + n_4$ 。依题意， $n_1 + 2n_2 + 3n_3 + 4n_4 = 10 + 2 + 30 + 80 = 122$ ， $n_1 + n_2 + n_3 + n_4 = 10 + 1 + 10 + 20 = 41$ ，可得出 $n_0 = 82$ ，即树 T 的叶结点的个数是 82。

10. C

解法 1：树有一个重要性质，即在 n 个结点的树中有 $n-1$ 条边，“那么对于每棵树，其结点数比边数多 1”。本题森林中的结点数比边数多 10（即 $25 - 15 = 10$ ），显然共有 10 棵树。

解法 2：仔细分析后发现此题也是考查图的性质：生成树和生成森林。对于图的生成树有一个重要的性质，即图中顶点数若为 n ，则其生成树含有 $n-1$ 条边。对比解法 1 中树的性质，不难发现两种解法都用到了性质“树中结点数比边数多 1”，后面的分析如解法 1。

5.2.4 答案与解析

一、单项选择题

01. C

在二叉树中,若某个结点只有一个孩子,则这个孩子的左右次序是确定的;而在度为2的有序树中,若某个结点只有一个孩子,则这个孩子就无须区分其左右次序,选项A错误。选项B仅当是完全二叉树时才有意义,对于任意一棵二叉树,高度可能为 $\lfloor \log_2 n \rfloor + 1 \sim n$ 。在完全二叉树中,若有度为1的结点,则只可能有一个,且该结点只有左孩子而无右孩子,选项C正确。完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$,也可以通过举例 $n=4$ 来排除,选项D错误。

02. C

“二叉树为空”意味着二叉树中没有结点,但并不意味着二叉树不存在。注意,线性表可以是空表,树可以是空树,但图不能是空图(图中不能没有结点)。

03. A

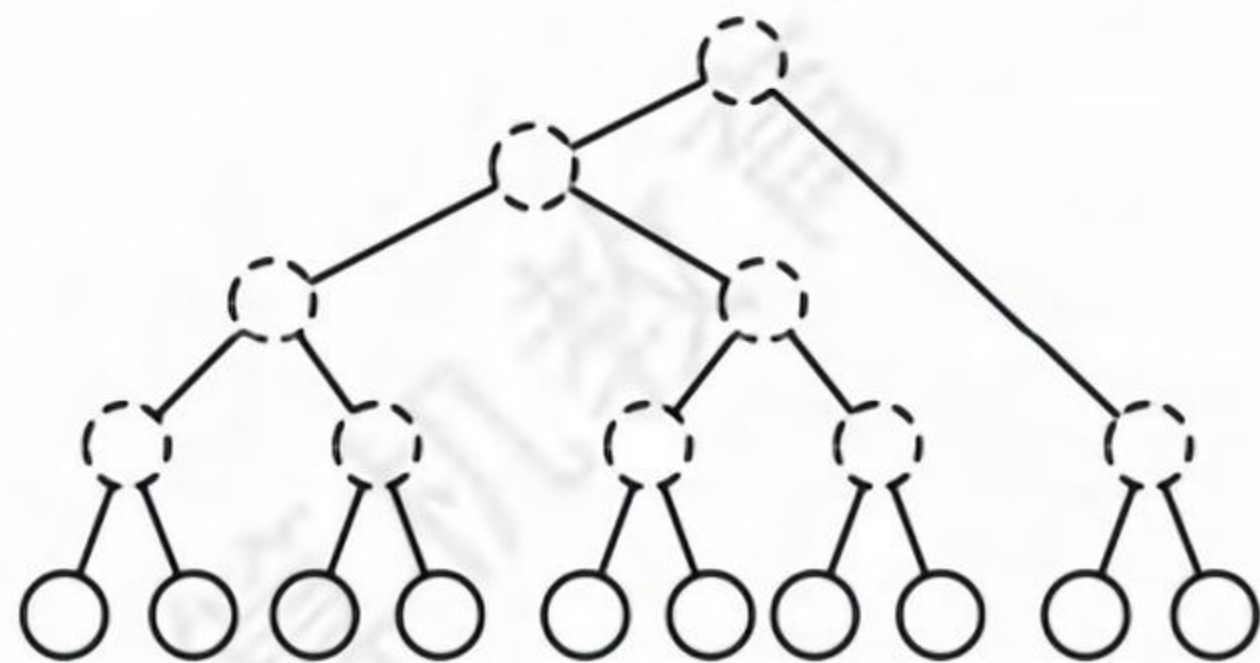
在完全二叉树中,叶结点的双亲的左兄弟的孩子一定在其前面(且一定存在),所以双亲的左兄弟(若存在)一定不是叶结点,选项A正确。 n_0 应等于 $n_2 + 1$,选项B错误。完全二叉树和满二叉树均可以采用顺序存储结构,选项C错误。第 i 个结点的左孩子不一定存在,选项D错误。

选项B的这种通用公式适用于所有二叉树,我们应能立即联想到采用特殊值代入法验证,如画一个只含3个结点的满二叉树的草图来验证是否满足条件。

04. B

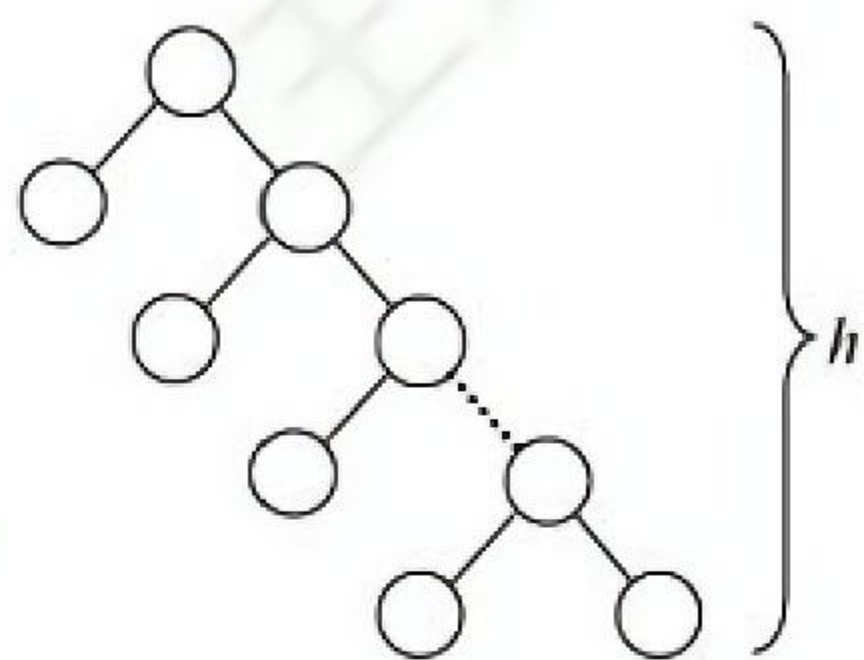
由二叉树的性质 $n_0 = n_2 + 1$,得 $n_2 = n_0 - 1 = 10 - 1 = 9$ 。

【另解】画出草图,如下图所示。首先画出10个叶结点,然后每2个结点向上合并,构造一个新的度为2的分支结点,直到构成如下图所示的二叉树,其中度为2的分支结点数为9。



05. B

结点最少的情況如下图所示。除根结点层只有 1 个结点外，其他 $h-1$ 层均有两个结点，结点总数 $= 2(h-1) + 1 = 2h-1$ 。



06. D

除根结点外，在其余 $n-1$ 个结点中，每个结点要么是其父结点的左孩子，要么是其父结点的右孩子，每个结点都有两种可能， $n-1$ 个结点共有 2^{n-1} 种不同的组合形态。

07. C

要求满足条件的树，分析可知当这 50 个结点构成一棵完全二叉树时高度最小， $h = \lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 50 \rfloor + 1 = 6$ 。

【另解】第 1 层最多有 1 个结点，第 2 层最多有 2^1 个结点，第 3 层最多有 2^2 个结点，第 4 层最多有 2^3 个结点，以此类推，可以得到 h 最少为 6。

08. C

由二叉树的性质 1 可知 $n_0 = n_2 + 1$ ，结点总数 $= 2n = n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$ ，则 $n_1 = 2(n - n_2) - 1$ ，所以 n_1 为奇数，说明该二叉树中不可能有 $2m$ 个度为 1 的结点。

09. C

当二叉树为单支树时具有最大高度，即每层上只有一个结点，最大高度为 1025。而当树为完全二叉树时，其高度最小，最小高度为 $\lfloor \log_2 n \rfloor + 1 = 11$ 。

10. C

建议画图，第一层有 1 个结点，其余 $h-1$ 层各有 2 个结点，总结点数 $= 1 + 2(h-1) = 15$ ， $h = 8$ 。

11. C

高度为 h 的完全二叉树中，第 1 层~第 $h-1$ 层构成一个高度为 $h-1$ 的满二叉树，结点个数为 $2^{h-1} - 1$ 。第 h 层至少有一个结点，所以最少的结点个数 $= (2^{h-1} - 1) + 1 = 2^{h-1}$ 。

12. A

第 6 层有叶结点说明完全二叉树的高度可能为 6 或 7，显然树高为 6 时结点最少。若第 6 层上有 8 个叶结点，则前 5 层为满二叉树，所以完全二叉树的结点个数最少为 $2^5 - 1 + 8 = 39$ 个结点。

13. A

深度为 6 的完全二叉树，第 5 层共有 $2^4 = 16$ 个结点。第 6 层最左边有 3 个叶结点，其对应的双亲结点为第 5 层最左边的两个结点，所以第 5 层剩余的结点均为叶结点，共有 $16 - 2 = 14$ 个，加上第 6 层的 3 个叶结点，共有 17 个叶结点。

14. D

由完全二叉树的性质，最后一个分支结点的序号为 $\lfloor 1001/2 \rfloor = 500$ ，所以叶结点个数为 501。

【另解】 $n = n_0 + n_1 + n_2 = n_0 + n_1 + (n_0 - 1) = 2n_0 + n_1 - 1$ ，因为 $n = 1001$ ，而在完全二叉树中， n_1 只能取 0 或 1。当 $n_1 = 1$ 时， n_0 为小数，不符合题意。所以 $n_1 = 0$ ，于是有 $n_0 = 501$ 。

15. C

要使二叉树第 7 层的结点数最多，只考虑树高为 7 层的情况，7 层满二叉树有 127 个结点，

126 仅比 127 少 1 个结点, 只能少在第 7 层, 所以第 7 层最多有 $2^6 - 1 = 63$ 个结点。

16. B

在非空的二叉树当中, 由度为 0 和 2 的结点数的关系 $n_0 = n_2 + 1$ 可知 $n_2 = 123$; 总结点数 $n = n_0 + n_1 + n_2 = 247 + n_1$, 其最大值为 248 (n_1 的取值为 1 或 0, 当 $n_1 = 1$ 时结点最多)。注意, 由完全二叉树总结点数的奇偶性可以确定 n_1 的值, 但不能根据 n_0 来确定 n_1 的值。

【另解】 $124 < 2^7 = 128$, 所以第 8 层没满, 前 7 层为完全二叉树, 由此可推算第 8 层可能有 120 个叶结点, 第 7 层的最右 4 个为叶结点, 考虑最多的情况, 这 4 个叶结点中的最左边可以有 1 个左孩子 (不改变叶结点数), 因此结点总数 $= 2^7 - 1 + 120 + 1 = 248$ 。

17. B

非空指针数 = 总分支数 $= n - 1$, 空指针数 $= 2 \times \text{结点总数} - \text{非空指针数} = 2n - (n - 1) = n + 1$ 。

【另解】在树中, 1 个指针对应 1 个分支, n 个结点的树共有 $n - 1$ 个分支, 即 $n - 1$ 个非空指针, 每个结点都有 2 个指针域, 所以空指针数 $= 2n - (n - 1) = n + 1$ 。

18. A

二叉链表表示的二叉树中空指针的数量为 $n + 1$, 三叉链表表示的二叉树多了一个根结点指向双亲的空指针, 所以树中空指针的数量为 $n + 2$, I 正确。若根结点的度为 2, 则只有左、右两个孩子指向它, II 错误。若整棵树只有一个根结点, 则没有指针指向它, III 错误。

19. A

由完全二叉树的性质, 编号为 i ($i \geq 1$) 的结点所在的层次为 $\lfloor \log_2 i \rfloor + 1$, 若两个结点位于同一层, 则一定有 $\lfloor \log_2 p \rfloor + 1 = \lfloor \log_2 q \rfloor + 1$, 因此有 $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor$ 成立。

20. C

当根结点下标为 1 时, 下标为 i 的结点的父结点下标为 $\lfloor i/2 \rfloor$, 那么下标为 17 的祖先的下标有 8, 4, 2, 1, 下标为 19 的祖先的下标有 9, 4, 2, 1, 因此两者最近的公共祖先的下标是 4。

21. C

分析可知, 满足条件的三叉树可以是完全三叉树, 这棵树的第 i ($i \geq 1$) 层最多有 3^{i-1} 个结点。设高度为 h , 则 $3^0 + 3^1 + \dots + 3^{h-1} = (3^h - 1)/2$ 是结点数上限, 问题是求解 $50 \leq (3^h - 1)/2$ 的最小 h 值, 即 $h \geq \log_3 101$, 有 $h = \lceil \log_3 101 \rceil = 5$ 。

22. B

三叉树采用三叉链表表示, 每个结点均有 3 个指针域指向 3 个孩子, 共有 $3n$ 个指针域, 但 n 个结点构成的一棵树中只需要 $n - 1$ 个指针 (对于 $n - 1$ 条边), 因此空指针域有 $2n + 1$ 个。

23. D

对于高度为 h 的满二叉树, $n = 2^0 + 2^1 + \dots + 2^{h-1} = 2^h - 1$, $m = 2^{h-1}$ 。

24. C

第 6 层有叶结点, 完全二叉树的高度可能为 6 或 7, 显然树高为 7 时结点最多。完全二叉树与满二叉树相比, 只是在最下一层的右边缺少部分叶结点, 而最后一层之上是个满二叉树, 且只有最后两层上有叶结点。若第 6 层上有 8 个叶结点, 则前 6 层为满二叉树, 而第 7 层缺失 $8 \times 2 = 16$ 个叶结点, 所以完全二叉树的结点个数最多为 $2^7 - 1 - 16 = 111$ 。

25. C

最后一个分支结点的编号为 $\lfloor 768/2 \rfloor = 384$, 所以叶结点的个数为 $768 - 384 = 384$ 。

【另解】 $n = n_0 + n_1 + n_2 = n_0 + n_1 + (n_0 - 1) = 2n_0 + n_1 - 1$, 其中 $n = 768$, 而在完全二叉树中, n_1 只能取 0 或 1, 当 $n_1 = 0$ 时, n_0 为小数, 不符合题意。因此 $n_1 = 1$, 所以 $n_0 = 384$ 。

26. A

非叶结点的度均为 2，且所有叶结点都位于同一层的完全二叉树就是满二叉树。对于一棵高度为 h 的满二叉树（空树 $h=0$ ），其最后一层全部是叶结点，数目为 2^{h-1} ；总结点数为 $2^h - 1$ 。因此当 $2^{h-1} = k$ 时，可以得到 $2^h - 1 = 2k - 1$ 。

27. A

二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为 5 的二叉树，为了满足任意性，其 1~5 层的所有结点都要被存储起来，即考虑为一棵高度为 5 的满二叉树，共需要存储单元的数量为 $1 + 2 + 4 + 8 + 16 = 31$ 。

28. C

高度一定的三叉树中结点数最多的情况是满三叉树。高度为 5 的满三叉树的结点数 $= 3^0 + 3^1 + 3^2 + 3^3 + 3^4 = 121$ ，高度为 6 的满三叉树的结点数 $= 3^0 + 3^1 + 3^2 + 3^3 + 3^4 + 3^5 = 364$ 。由于三叉树 T 的结点数为 244， $121 < 244 < 364$ ，因此 T 的高度至少为 6。

5.3.4 答案与解析

一、单项选择题

01. C

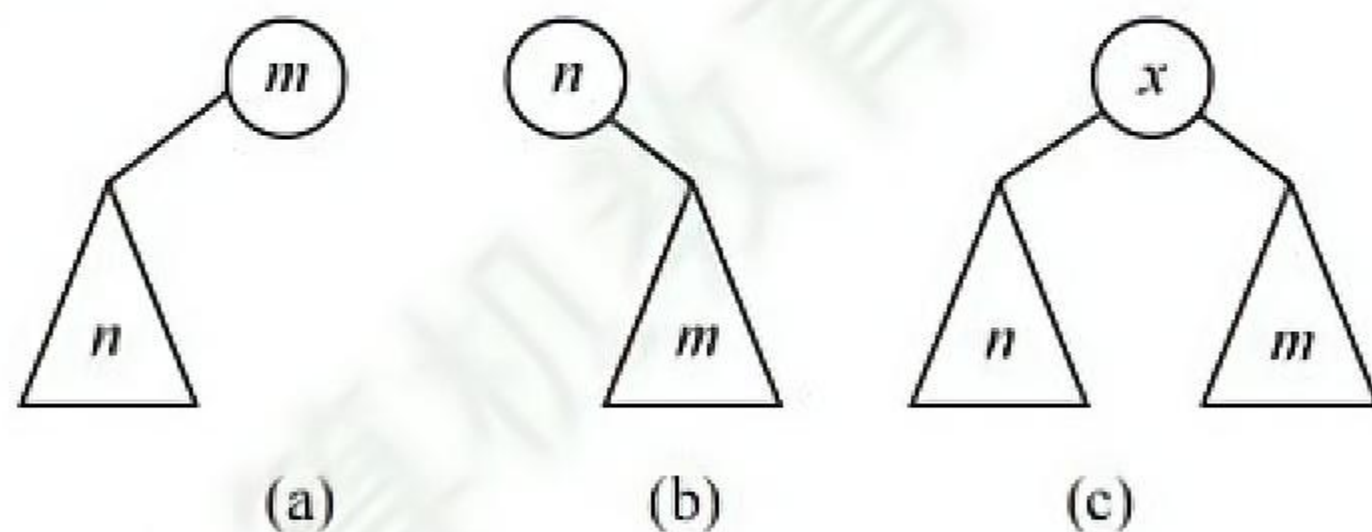
二叉树中序遍历的最后一个结点一定是从根开始沿右子女指针链走到底的结点, 设用 p 指示。若结点 p 不是叶结点 (其左子树非空), 则前序遍历的最后一个结点在它的左子树中, A、B 错误; 若结点 p 是叶结点, 则前序与中序遍历的最后一个结点就是它, C 正确。若中序遍历的最后一个结点 p 不是叶结点, 它还有一个左子女 q , 结点 q 是叶结点, 那么结点 q 是前序遍历的最后一个结点, 但不是中序遍历的最后一个结点, D 错误。

02. C

三种遍历方式中, 都先遍历左子树, 再遍历右子树, 因此 b 一定在 c 的前面访问。

03. C

中序遍历时, 先访问左子树, 再访问根结点, 后访问右子树。 n 在 m 前的 3 种可能性如下图所示, 从中看出 n 总是在 m 的左方。



【另解】设 n 和 m 的最近公共祖先 p ，则有以下可能：

情形 1， m 和 n 分别在 p 的左、右（右、左）分支上；情形 2， m 或 n 为 p 结点，另一结点在 p 的分支上。只有 n 和 m 分别处于 p 的左、右分支上， m 为祖先结点且 n 位于 m 的左分支， n 为祖先结点且 m 位于 n 的右分支，符合题意。

04. D

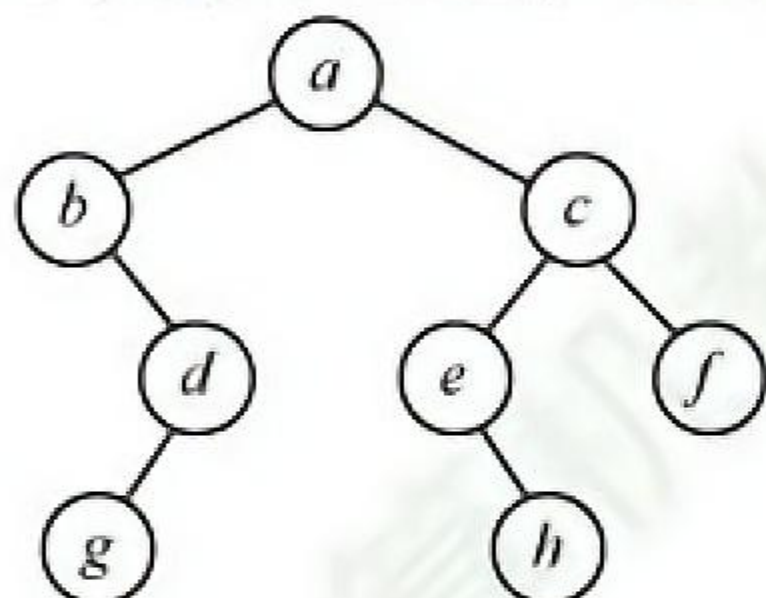
后序遍历的顺序是 LRN，若 n 在 N 的左子树， m 在 N 的右子树，则在后序遍历的过程中 n 在 m 之前访问；若 n 是 m 的子孙，设 m 在 N 的位置，则 n 无论是在 m 的左子树还是在右子树，在后序遍历的过程中 n 都在 m 之前访问。其他都不可以。选项 C 要成立，就需加上两个结点位于同一层这个条件。

05. C

在后序遍历退回时访问根结点，就可以从下向上把从 n 到 m 的路径上的结点输出，若采用非递归的算法，则当后序遍历访问到 n 时，栈中把从根到 n 的父指针的路径上的结点都记忆下来，也可以找到从 m 到 n 的路径。

06. C

在二叉树的数组存储结构中，下标为 i 的结点的左、右孩子的下标分别为 $2i+1$ 和 $2i+2$ （若存在），画出二叉树的形态如下图所示，则后序遍历序列为 $gdbhefca$ 。



07. B

三种遍历方式中，访问左、右子树的先后顺序是不变的，只是访问根结点的顺序不同，因此叶结点的先后顺序完全相同。此外，读者可以采用特殊值法，画一个结点数为 3 的满二叉树，采用三种遍历方式来验证答案的正确性。

08. C

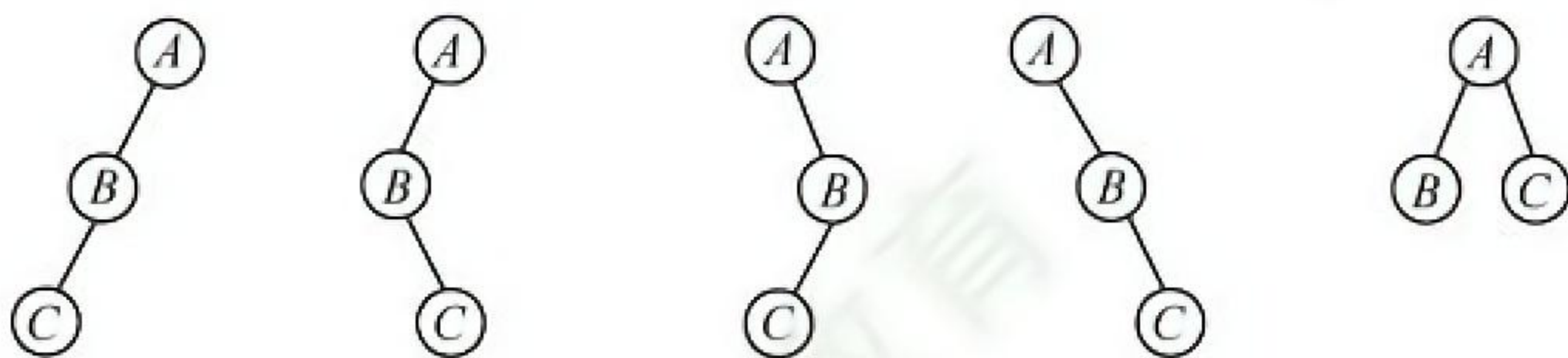
对每个顶点从 1 开始按序编号，要求结点编号大于其左、右孩子编号，并且左孩子编号小于右孩子编号。编号越大说明遍历顺序越靠后，因此，三者遍历顺序为先左子树、再右子树、后根结点。4 个选项中仅后序遍历满足要求。

09. B

结点 v 的编号比其左子树上的最小编号还小，而 v 的右子树中的最小编号大于 v 的左子树中的最大编号，因此 v 的编号比其左、右子树上的所有编号都小，显然是按先序遍历次序。

10. D

前序为 A 、 B 、 C 的不同二叉树共有 5 种，其中后序为 C 、 B 、 A 的有 4 种（前 4 种），都是单支树，如下图所示。



11. C

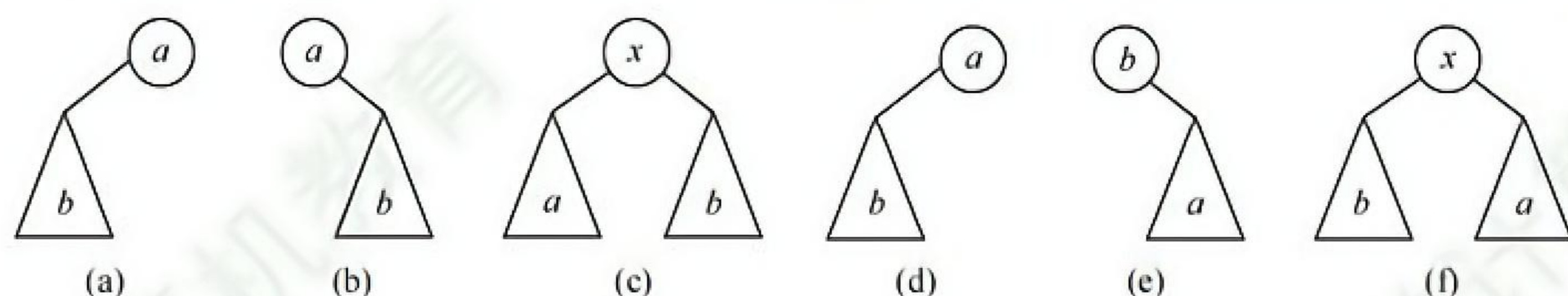
7 个结点的完全二叉树是一棵 3 层的满二叉树，画出相应二叉树的树形，根据后序遍历序列填入相应的结点，得到相应的完全二叉树，求得其先序遍历序列为 $ABCDEFG$ 。

12. C

二叉树的前序遍历为 NLR，后序遍历为 LRN。根据题意，在前序序列中 X 在 Y 之前，在后序序列中 X 在 Y 之后，若设 X 在根的位置， Y 在其左子树或右子树中，即满足要求。

13. C

先序序列是 $\dots a \dots b \dots$ ，因此 a 和 b 结点的 3 种情况如下图(a)~(c)所示。中序序列是 $\dots b \dots a \dots$ ，因此 a 和 b 结点的 3 种情况如下图(d)~(f)所示，相同部分是 b 在 a 的左子树中。



14. B

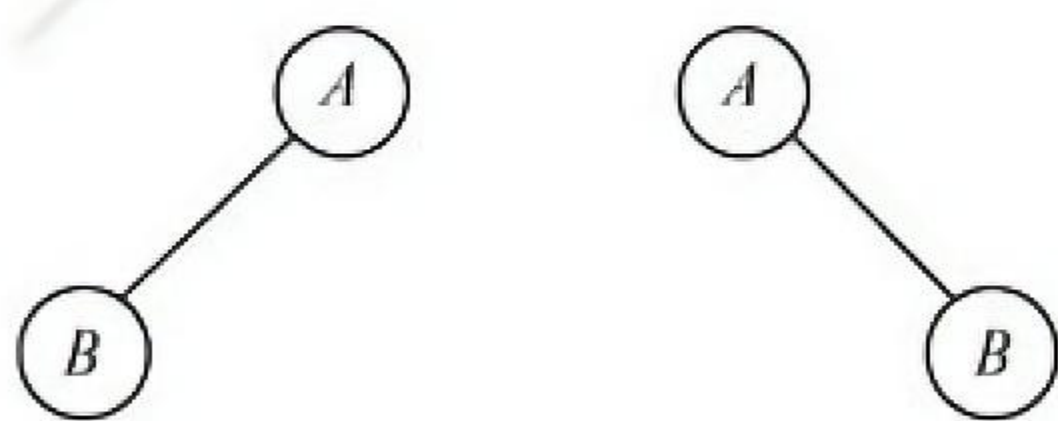
解法 1：由题可知，1 为根结点，2 为 1 的孩子。对于 A，3 应为 1 的左孩子，前序序列应为 13...，不符题意。类似的，C 也错误。对于 B，2 为 1 的右孩子，3 为 2 的右孩子……满足题意。对于 D，463572 应为 1 的右子树，2 为 1 的右孩子，46357 为 2 的左子树，3 为 2 的左孩子，46 为 3 的左子树，57 为 3 的右子树，前序序列 4、6 应相连，5、7 应相连，不符题意。

解法 2：前序遍历时需要借助栈。二叉树的前序序列和中序序列的关系相当于以前序序列作为入栈次序，以中序序列作为出栈次序。题中以 1234567 入栈；对于 A，第一个出栈的是 3，所以 1 不可能在 2 之前出栈，错误。对于 C，1 不可能在 3 之前出栈，错误。对于 D，6 第三个出栈，此时栈顶元素是 5，不是 3，错误。B 正确。

解法 3：因前序序列和中序序列可以确定一棵二叉树，所以可试着用题目中的序列构造出相应的二叉树，即可得知，只有选项 B 的序列可以构造出二叉树。

15. D

先序序列为 NLR，后序序列为 LRN，虽然可以唯一确定树的根结点，但无法划分左、右子树。例如，先序序列为 AB，后序序列为 BA，则其对应的二叉树如下图所示。

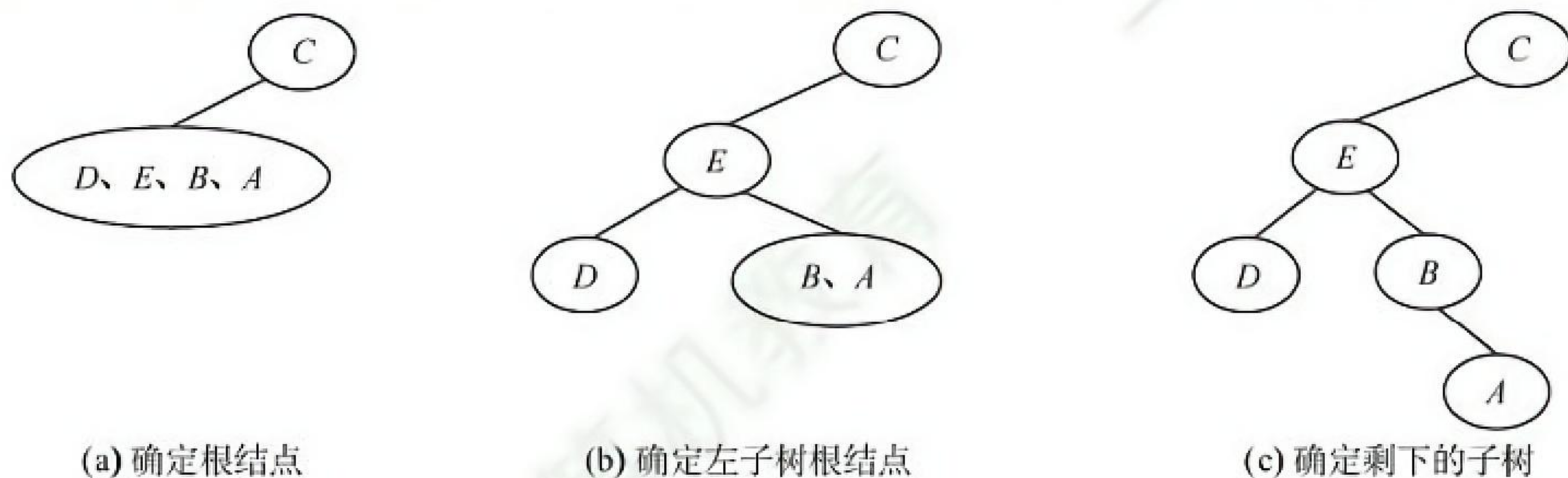


16. B

中序遍历是“左根右”，后序遍历是“左右根”，当任一结点没有右子树时，两种遍历都是“左根”。显然，当二叉树为空树或只有根结点时，其中序序列和后序序列也相同。

17. D

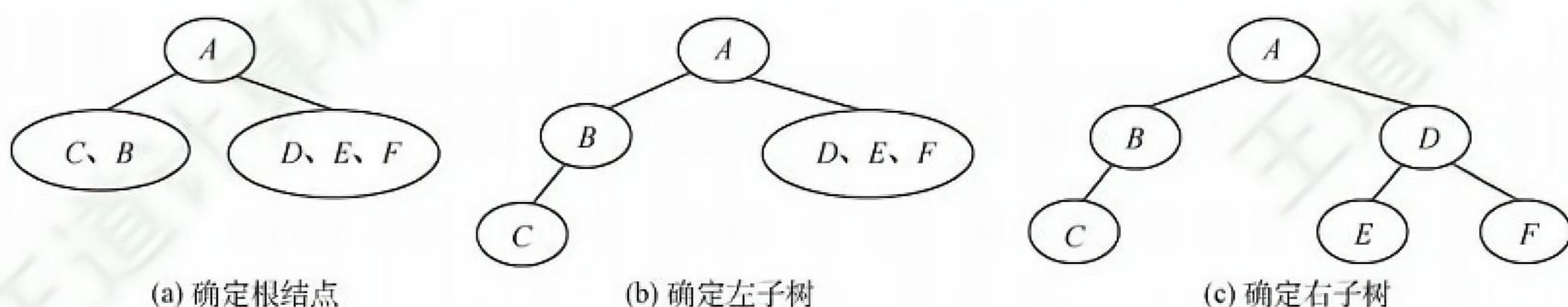
根据后序序列与中序序列可构造出二叉树，如下图所示。由图可知先序序列为 CEDBA。



18. A

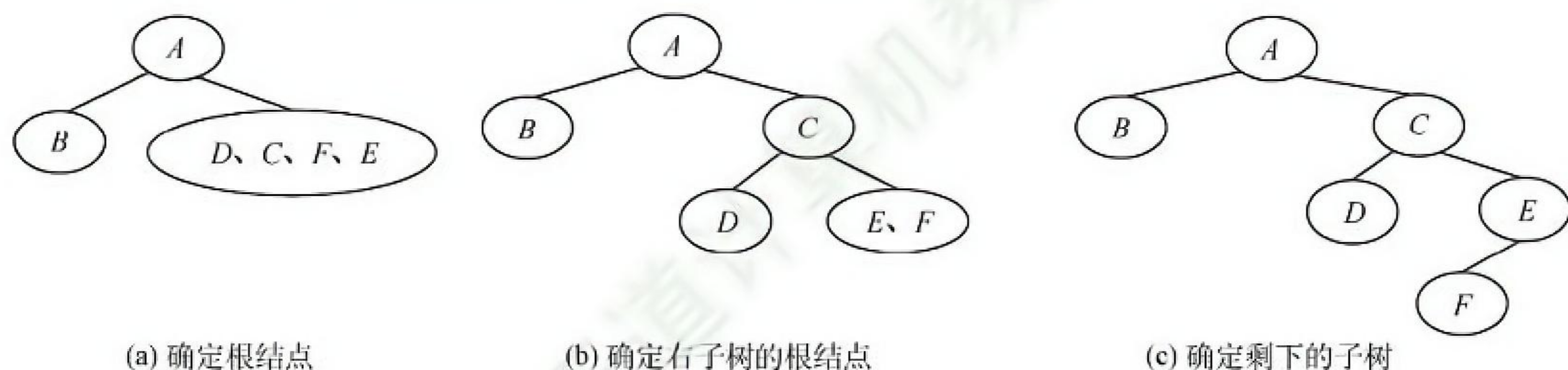
对于这种遍历序列问题,先根据遍历的性质排除若干项,若还无法确定答案,则再根据遍历结果得到二叉树,找到对应遍历序列。例如,在本题中,已知先序和中序遍历结果,可知本树的根结点为 A ,左子树有 C 和 B ,其余为右子树,则后序遍历结果中, A 一定在最后,并且 C 和 B 一定在前面,排除答案 B 和 D 。又因先序中有 DEF ,中序中有 EDF ,则 D 为这个子树的根,所以 D 在后序中排在 EF 之后。

根据二叉树的递归定义,要确定二叉树,就要分别找到根结点和左、右子树。因此,根据遍历结果,必定要确定根结点位置和如何划分左、右子树,才可以确定最终的二叉树。因此,仅有先序和后序遍历不能唯一确定一棵二叉树,而二者之一加上中序遍历都可以唯一确定一棵二叉树。如在本题中,根据先序和中序遍历的结果确定二叉树的过程如下图所示。



19. B

可构造出二叉树如下图所示。因此,先序序列为 $ABCDEF$ 。



20. B

若 a 是 b 的祖先,则后序遍历时一定先遍历 b 后遍历 a ,所以 B 错误。

21. C

删除一个结点时,需要先递归地删除它的左、右孩子,并释放它们所占的存储空间,然后删除该结点,并删除它所占的存储空间,这正好和后序遍历的访问顺序相吻合。

22. A

线索是前驱结点和后继结点的指针,引入线索的目的是加快对二叉树的遍历。

23. C

二叉树是一种逻辑结构,但线索二叉树是加上线索后的链表结构,即它是二叉树在计算机内部的一种存储结构,所以是一种物理结构。

24. C

n 个结点共有链域指针 $2n$ 个,其中,除根结点外,每个结点都被一个指针指向。剩余的链域建立线索,共 $2n - (n - 1) = n + 1$ 个线索。

25. C

线索二叉树中用 $ltag/rtag$ 标识结点的左/右指针域是否为线索,其值为 1 时,对应指针域为线索,其值为 0 时,对应指针域为左/右孩子。

26. D

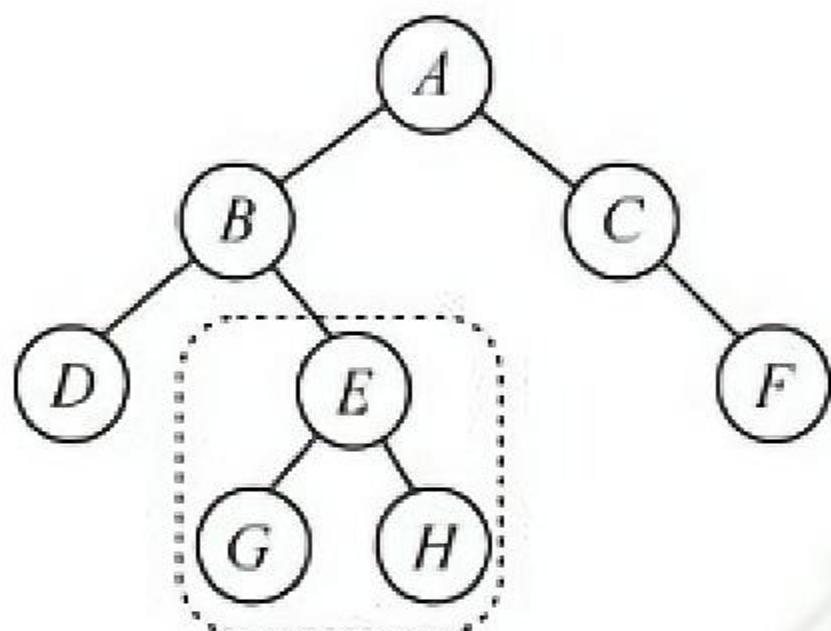
对左子树为空的二叉树进行先序线索化, 根结点的左子树为空并且也没有前驱结点(先遍历根结点), 先序遍历的最后一个元素为叶结点, 左、右子树均为空且有前驱无后继结点, 所以线索化后, 树中空链域有 2 个。

27. D

不是每个结点通过线索都可以直接找到它的前驱和后继。在先序线索二叉树中查找一个结点的先序后继很简单, 而查找先序前驱必须知道该结点的双亲结点。同样, 在后序线索二叉树中查找一个结点的后序前驱也很简单, 而查找后序后继也必须知道该结点的双亲结点, 二叉链表中没有存放双亲的指针。

28. D

后序线索二叉树不能有效解决求后序后继的问题。如下图所示, 结点 E 的右指针指向右孩子, 而在后序序列中 E 的后继结点为 B , 在查找 E 的后继时仍然只能按常规方法来查找。



29. C

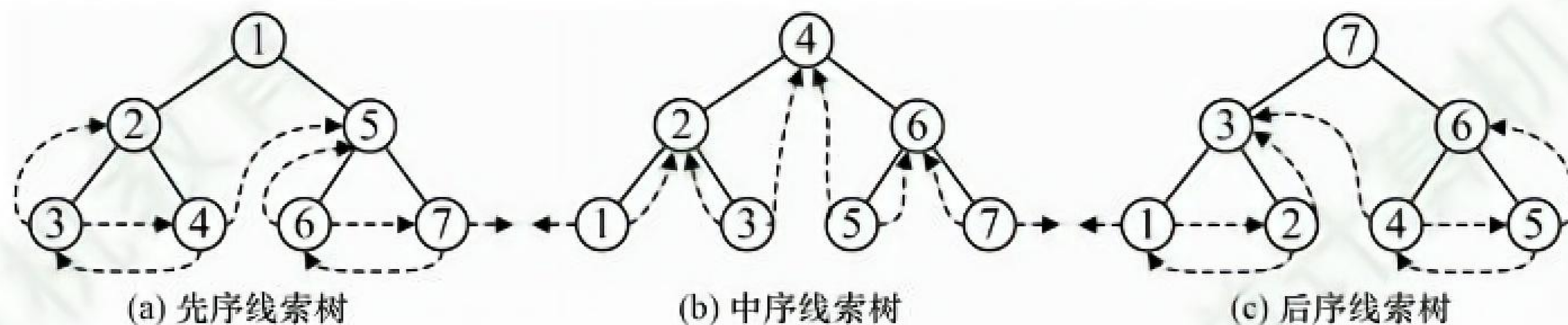
在二叉中序线索树中, 某结点若有左孩子, 则按照中序“左根右”的顺序, 该结点的前驱结点为左子树中最右的一个结点(注意, 并不一定是最右叶结点)。

30. A

在二叉树的后序遍历中, 叶结点 X 的后继是其双亲, 因此 X 的右线索应指向该结点。

31. C

后序线索树遍历时, 最后访问根结点, 若从右孩子 x 返回访问父结点, 则由于结点 x 的右孩子不一定为空(右指针无法指向其后继), 因此通过指针可能无法遍历整棵树。如下图所示, 结点中的数字表示遍历的顺序, 图(c)中结点 6 的右指针指向其右孩子 5, 而不指向其后序后继结点 7, 因此后序遍历还需要栈的支持, 而图(a)和图(b)均可遍历。



32. B

非空二叉树的先序序列和后序序列相反, 即“根左右”与“左右根”顺序相反, 因此树只有根结点, 或根结点只有左子树或右子树, 其子树也有同样的性质, 任意结点只有一个孩子, 才能满足先序序列和后序序列正好相反。此时树形应为一个长链, 树中仅有一个叶结点。

33. D

分析遍历后的结点序列, 可以看出根结点是在中间被访问的, 而且右子树结点在左子树之前, 则遍历的方法是 RNL。本题考查的遍历方法并不是二叉树遍历的 3 种基本遍历方法, 对于考生而言, 重要的是掌握遍历的思想。

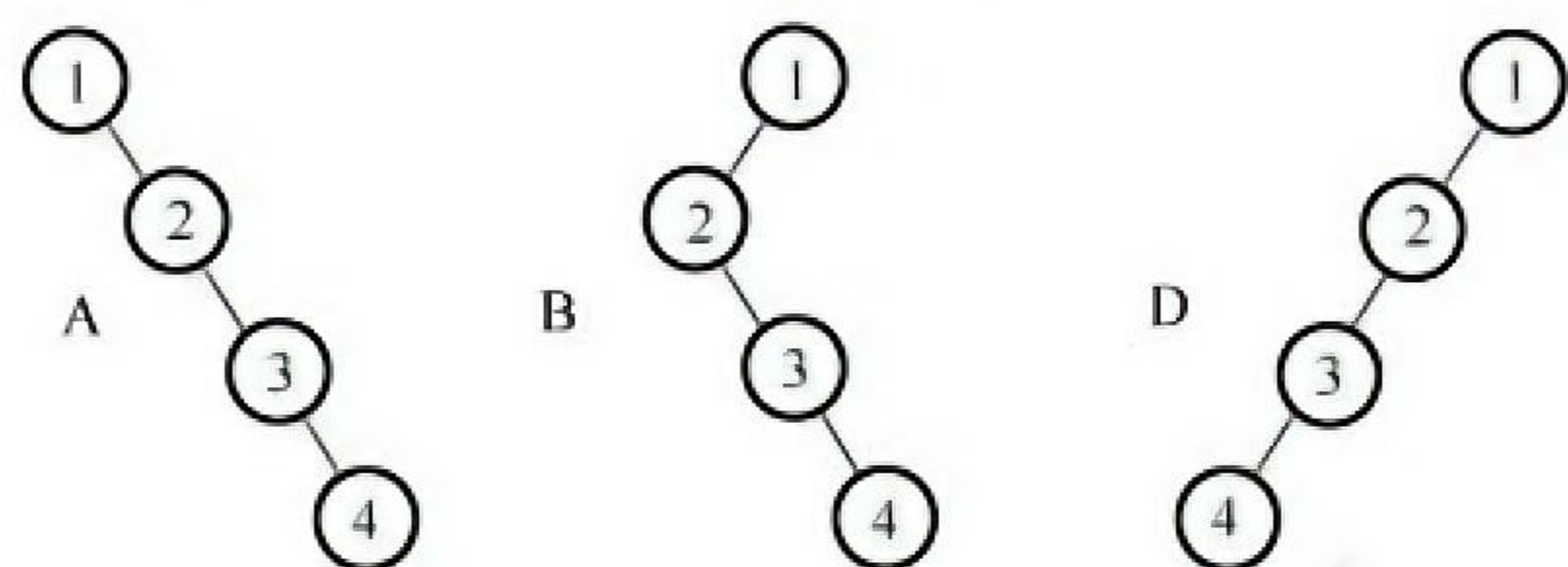
34. D

题中所给二叉树的后序序列为 $dbca$ 。结点 d 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 b ；结点 b 无左子树，左链域指向其前驱结点 d ；结点 c 无左子树，左链域指向其前驱结点 b ，无右子树，右链域指向其后继结点 a 。

35. C

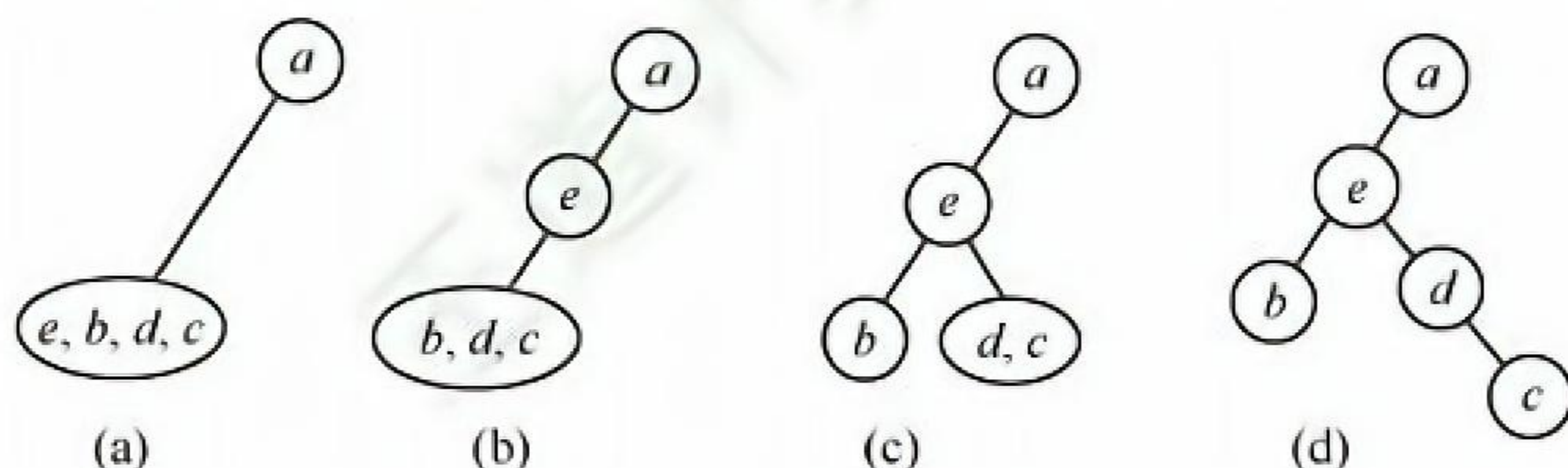
前序序列为 NLR ，后序序列为 LRN ，由于前序序列和后序序列刚好相反，所以不可能存在一个结点同时有左右孩子，即二叉树的高度为 4。1 为根结点，由于根结点只能有左孩子（或右孩子），因此在中序序列中，1 或在序列首或在序列尾，选项 A、B、C、D 皆满足要求。仅考虑以 1 的孩子结点 2 为根结点的子树，它也只能有左孩子（或右孩子），因此在中序序列中，2 或在序列首或在序列尾，选项 A、B、D 皆满足要求。

【另解】画出各选项与题干信息所对应的二叉树如下，所以 A、B、D 均满足。



36. A

前序序列和后序序列不能唯一确定一棵二叉树，但可以确定二叉树中结点的祖先关系：当两个结点的前序序列为 XY 与后序序列为 YX 时，则 X 为 Y 的祖先。考虑前序序列 a, e, b, d, c 、后序序列 b, c, d, e, a ，可知 a 为根结点， e 为 a 的孩子结点；此外，由 a 的孩子结点的前序序列 e, b, d, c 、后序序列 b, c, d, e ，可知 e 是 bcd 的祖先，所以根结点的孩子结点只有 e 。



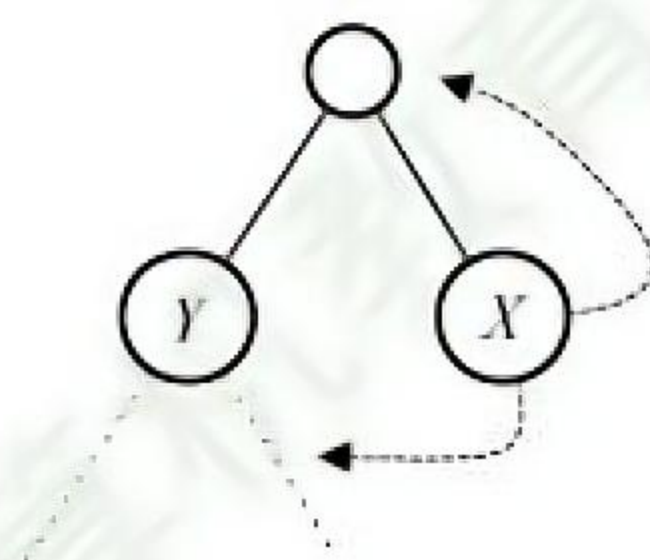
排除法：显然 a 为根结点，且确定 e 为 a 的孩子结点，排除 D。各种遍历算法中左右子树的遍历次序是固定的，若 b 也为 a 的孩子结点，则在前序序列和后序序列中 e, b 的相对次序应是不变的，所以排除 B，同理排除 C。

特殊法：前序序列和后序序列对应多棵不同的二叉树树形，我们只需画出满足该条件的任意一棵二叉树即可，任意一棵二叉树必定满足正确选项的要求。

显然选择 A，最终得到的二叉树满足题设中前序序列和后序序列的要求。

37. A。

根据后序线索二叉树的定义， X 结点为叶结点且有左兄弟，因此这个结点为右孩子结点，利用后序遍历的方式可知 X 结点的后序后继是其父结点，即其右线索指向的是父结点。为了更加形象，在解题的过程中可以画出如下所示的草图。



38. D

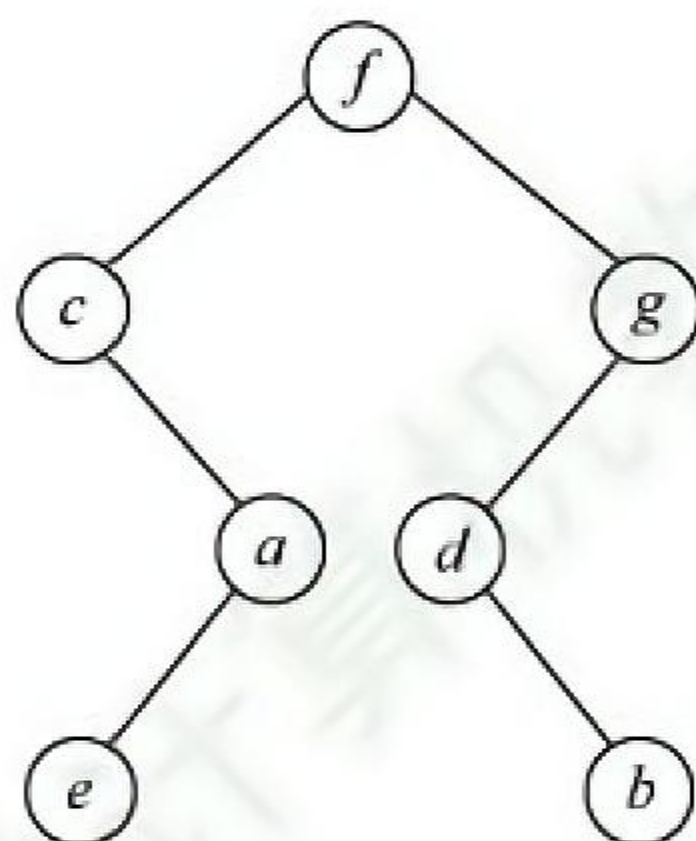
线索二叉树的线索实际上指向的是相应遍历序列特定结点的前驱结点和后继结点, 所以先写出二叉树的中序遍历序列 $debxac$, 中序遍历中在 x 左边和右边的字符, 就是它在中序线索化的左、右线索, 即 b, a 。

39. B

根据二叉树前序遍历和中序遍历的递归算法中递归工作栈的状态变化得出: 前序序列和中序序列的关系相当于以前序序列为入栈次序, 以中序序列为出栈次序。因为前序序列和中序序列可以唯一地确定一棵二叉树, 所以题意相当于“以序列 a, b, c, d 为入栈次序, 则出栈序列的个数为?”, 对于 n 个不同元素进栈, 出栈序列的个数为 $\frac{1}{n+1}C_{2n}^n = 14$ 。

40. B

后序序列先访问左子树, 接着访问右子树, 最后访问父结点, 递归进行。根结点左子树的叶结点首先被访问, 它是 e 。接下来是它的父结点 a , 然后是 a 的父结点 c 。接着访问根结点的右子树。它的叶结点 b 首先被访问, 然后是 b 的父结点 d , 再后是 d 的父结点 g , 最后是根结点 f , 如下图所示。因此 d 与 a 同层, B 正确。

**41. B**

先序序列先访问父结点, 接着访问左子树, 然后访问右子树。中序序列先访问左子树, 接着访问父结点, 然后访问右子树, 递归进行。若所有非叶结点只有右子树, 则先序序列和中序序列都先访问父结点, 后访问右子树, 递归进行。

42. B

对于此类题, 每种情况只需举出一个反例即可。如图 1 所示, q 是 p 的双亲, 中序遍历序列为 $\{p, q\}$, 选项 I 可能。如图 2 所示, q 是 p 的右孩子, 中序遍历序列为 $\{p, q\}$, 选项 II 可能。如图 4 所示, q 是 p 的双亲的双亲, 中序遍历序列为 $\{x, p, q\}$, 选项 IV 可能。如图 3 所示, q 是 p 的右兄弟, F 是 q 和 p 的父结点, 中序遍历要求先遍历左子树, 再访问根结点, 最后遍历右子树, 因此一定先访问 p , 再访问 F , 最后访问 q , p 和 q 不可能相邻出现, 选项 III 不可能。

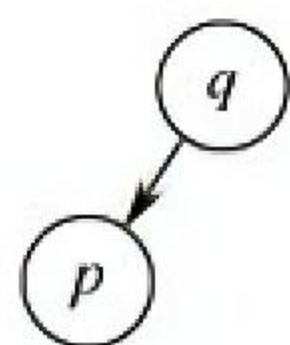


图 1

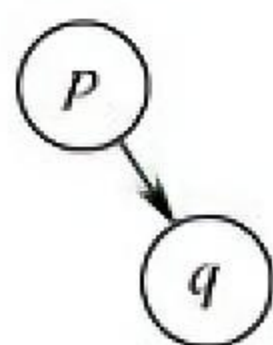


图 2

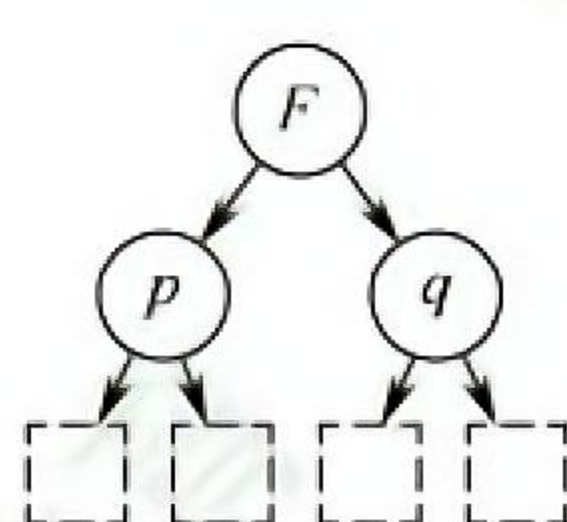


图 3

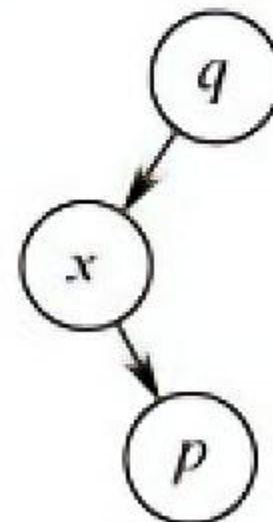
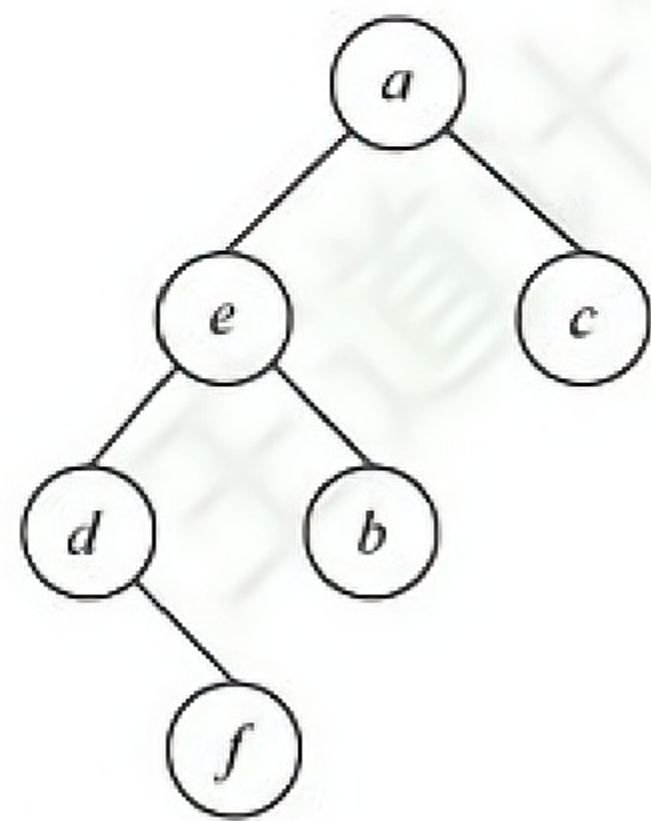


图 4

43. A

根据二叉树的树形和后序遍历序列, 可以轻松地将各字母填入结点中, 如下图所示。



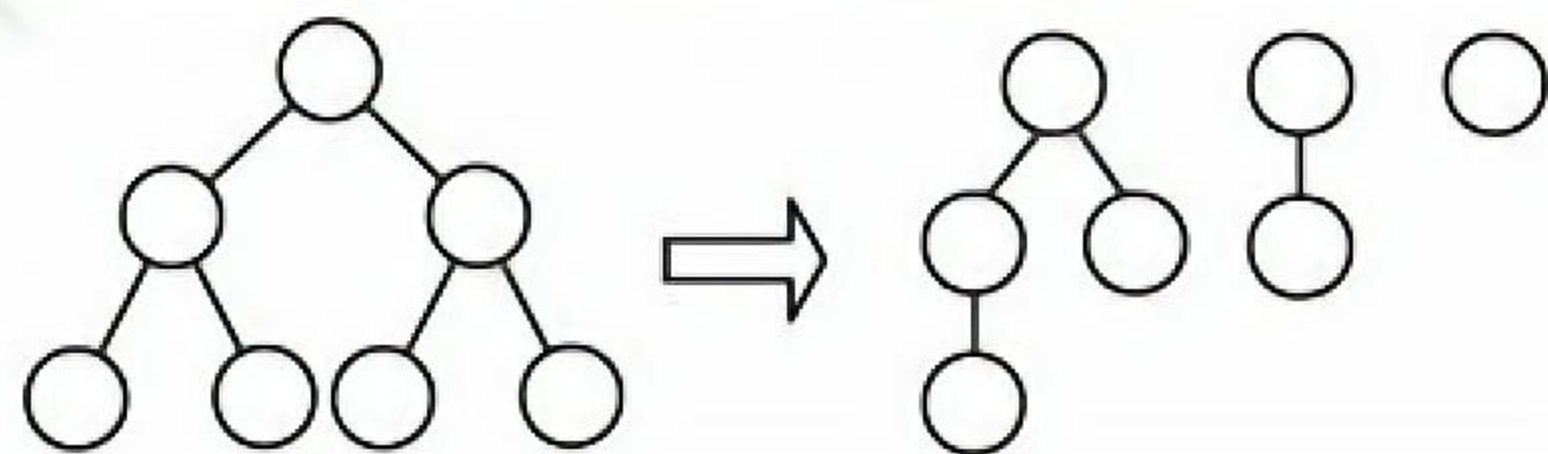
然后对该二叉树进行先序遍历，得到序列 $aedfbc$ 。

5.4.5 答案与解析

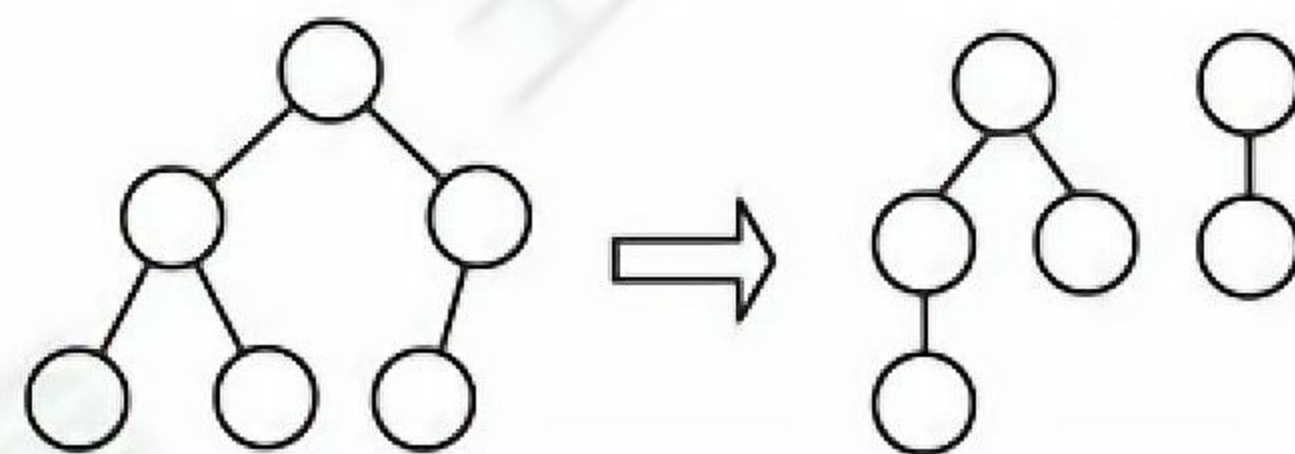
一、单项选择题

01. D

若 n 个结点的二叉树是一棵单支树，则其高度为 n 。完全二叉树中最多存在一个度为 1 的结点且只有左孩子，若不存在左孩子，则一定也不存在右孩子，因此必是叶结点，选项 II 正确。只有满二叉树才具有性质 III，如下图所示。



(a) 满二叉树转化为对应的森林



(b) 非满二叉树转化为对应的森林

在树转换为二叉树时，若有几个叶结点具有共同的双亲，则转换成二叉树后只有一个叶结点（最右边的叶结点），如下图所示，选项 IV 错误。注意，若树中的任意两个叶结点都不存在相同的双亲，则树中的叶子数才有可能与其对应的二叉树中的叶子数相等。

**02. D**

森林与二叉树具有对应关系, 因此, 我们存储森林时应先将森林转换成二叉树, 转换的方法就是“左孩子右兄弟”, 与树不同的是, 若存在第二棵树, 则二叉链表的根结点的右指针指向的是森林中的第二棵树的根结点。若此森林只有一棵树, 则根结点的右指针为空。因此, 右指针可能为空也可能不为空。

03. D

与树转换为二叉树不同, 森林中的每棵树是独立的, 因此先要将每棵树的根结点全部视为兄弟结点的关系。森林转换为二叉树后, 树 2 作为树 1 的根结点的右子树, 树 3 作为树 2 的根结点的右子树, 因此森林 F 对应的二叉树根结点的右子树上的结点个数是 $M_2 + M_3$ 。

04. C

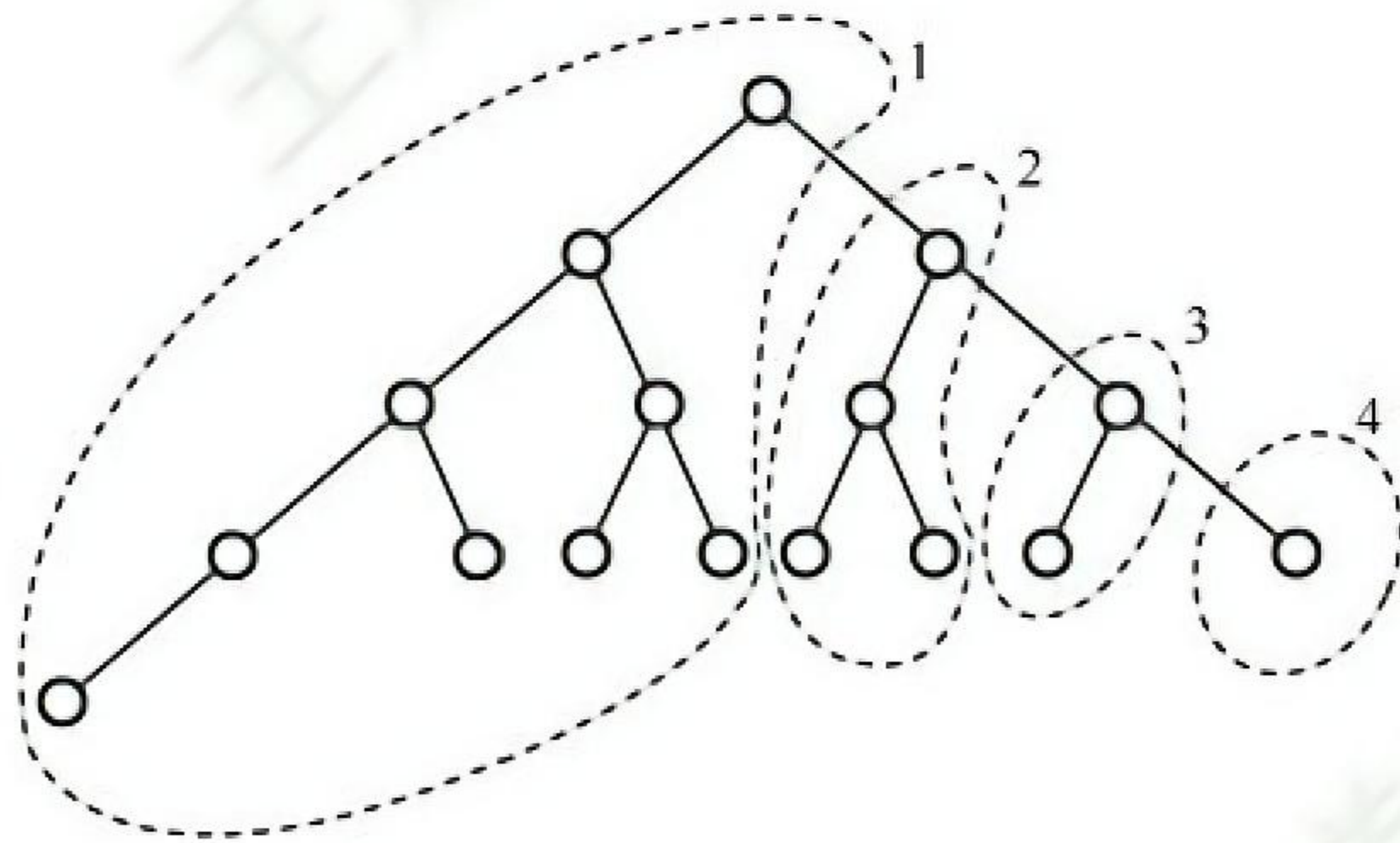
森林转换为二叉树后, 二叉树的根结点为第 1 棵树的根结点, 二叉树的根结点的左子树包含第 1 棵树的所有孩子, 因此森林 F 对应的二叉树的根结点的左子树上的结点数是 $a - 1$ 。

05. A

森林转换成二叉树时采用孩子兄弟表示法, 根结点及其左子树为森林中的第一棵树。右子树为其他剩余的树。所以, 第一棵树的结点个数为 $m - n$ 。

06. D

森林转换为二叉树后, 二叉树的根结点及其左子树由第 1 棵树转换得到, 二叉树的根结点的右子树由剩余的森林转换得到, 以此类推, 可以划分出第 2, 3, ... 棵树的结点。具有 16 个结点的完全二叉树的形态如下图所示, 沿二叉树的根结点往右下遍历, 共有 4 个结点, 可知森林中有 4 棵树, 其中第 1 棵树的结点数最多, 有 9 个。

**07. B**

将森林中每棵树的根结点视为兄弟结点的关系, 再按照“左孩子右兄弟”的规则来进行转化。

08. C

根据森林与二叉树转换规则“左孩子右兄弟”。二叉树 B 中右指针域为空代表该结点没有兄弟结点。森林中每棵树的根结点从第二个开始依次连接到前一棵树的根的右孩子, 因此最后一棵树的根结点的右指针为空。另外, 每个非终端结点, 其所有孩子结点在转换之后, 最后一个孩子的右指针也为空, 所以树 B 中右指针域为空的结点有 $n + 1$ 个。

09. B

在树的孩子兄弟表示法中, 若一个结点没有孩子 (即叶结点), 则表现为该结点的左指针域

为空, 因此本题答案为“6”。至于“5个结点的左、右指针域都为空”, 表示树中有5个结点既没有孩子又没有兄弟, 约束条件比题中的“求叶结点的个数”要求更严格。

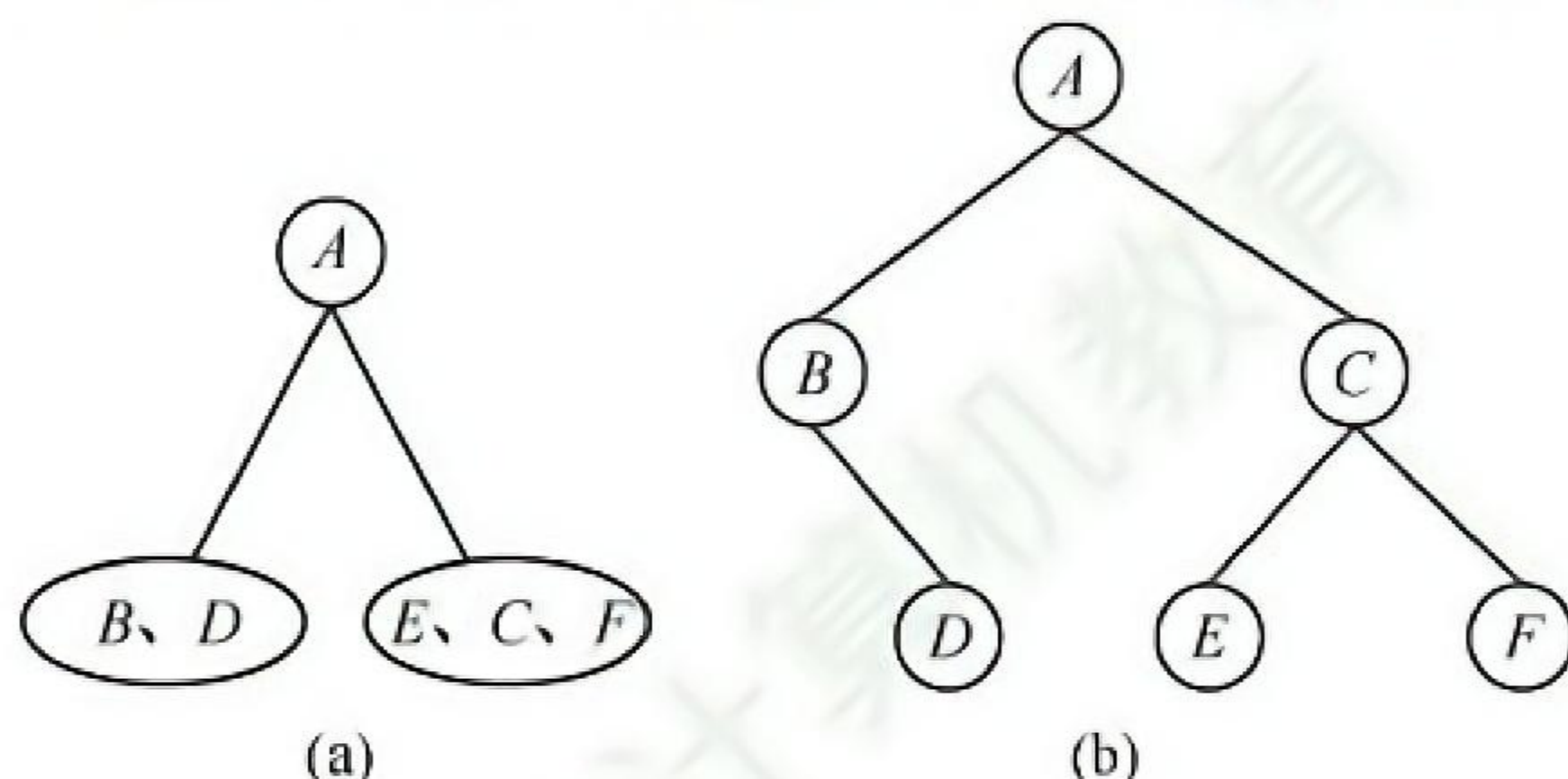
10. B

有序树 T 转换成二叉树 T_1 时, T 的后根序列是对应 T_1 的中序序列还是后序序列呢(显然树的后根序列不可能对应二叉树的先序序列和层序序列)? 看下图所示的例子, 在树 T 中, 叶结点 B 应最先访问, 在 T_1 中, B 的右兄弟 C 转换为它的右孩子, 若对应 T_1 的后序序列, 则 C 应在 B 的前面访问, 所以 T 的后根序列不可能对应 T_1 的后序序列。



11. C

根据二叉树的前序序列和中序序列可以确定一棵二叉树。根据后序序列, A 是二叉树的根结点。根据中序序列, 二叉树的形态如下图(a)所示。对于 A 的左子树, 根据后序序列, B 比 D 后被访问, 因此 B 必为 D 的父结点, 又根据中序序列, D 是 B 的右儿子。对于 A 的右子树, 同理可确定结点 E 、 C 、 F 的关系。此二叉树的形态如下图(b)所示。



再根据二叉树与森林的对应关系, 森林中树的棵数即为其对应二叉树(向右上旋转 45° 后)的根结点 A 及其右兄弟数, 或解释为: 对应二叉树从根结点 A 开始不断往右孩子访问, 所访问到的结点数。可知此森林中有3棵树, 根结点分别为 A 、 C 和 F 。

12. D

在二叉树 B 中, X 是其双亲的右孩子, 因此在树 T 中, X 必是其双亲结点的右兄弟, 换句话说, X 在树中必有左兄弟。

13. B

在森林的二叉树表示中, 当 M 和 N 的父结点是二叉树根结点时, M 和 N 在不同的树上。因此 M 和 N 可能无公共祖先。

14. B

森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中, 父子关系在对应森林关系中可能是兄弟关系或者原本就是父子关系。

情形 I: 若结点 v 是结点 u 的第二个孩子结点, 转换时, 结点 v 就变成结点 u 第一个孩子的右孩子, 符合要求。情形 II: 结点 u 和 v 是兄弟结点的关系, 但二者之中还有一个兄弟结点 k , 则转换后结点 v 就变为结点 k 的右孩子, 而结点 k 则是结点 u 的右孩子, 符合要求。

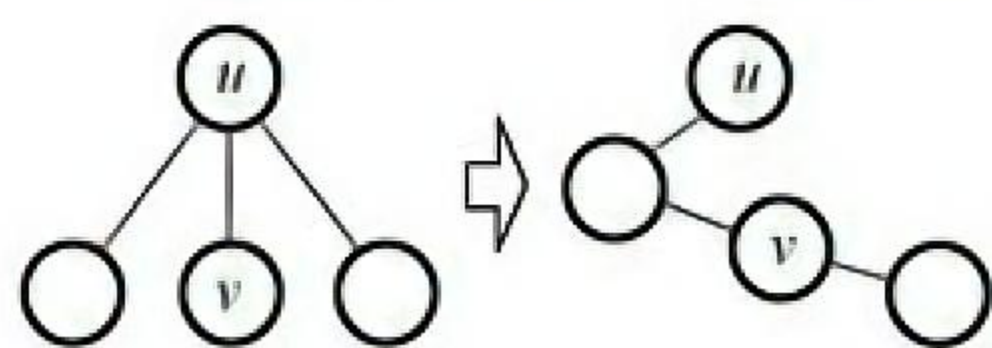


图 I

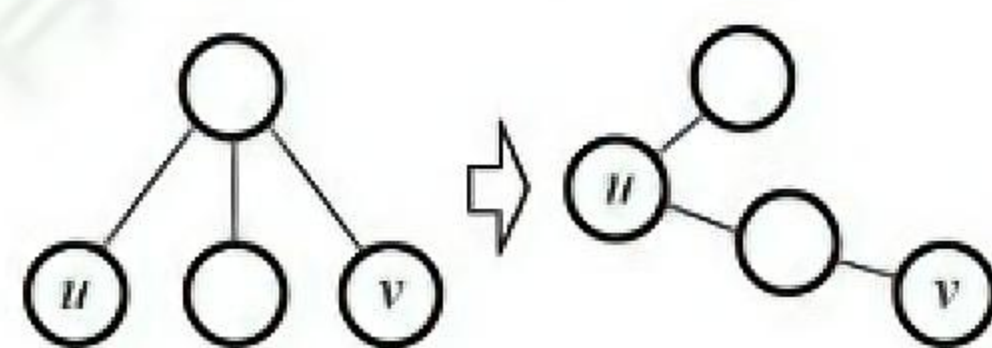


图 II

情形 III: 若结点 u 的父结点与 v 的父结点是兄弟关系, 则转换后, 结点 u 和 v 分别在两者最左父结点的两棵子树中, 不可能出现在同一条路径中。

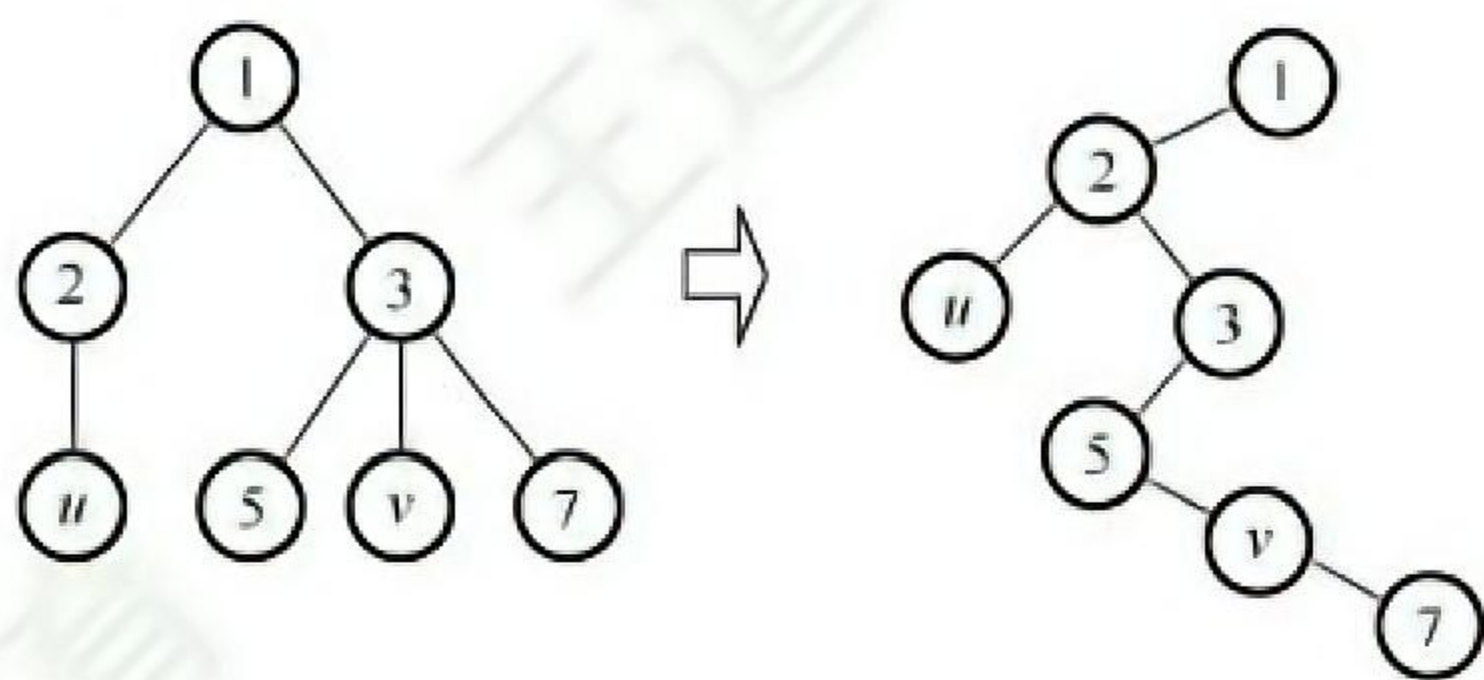
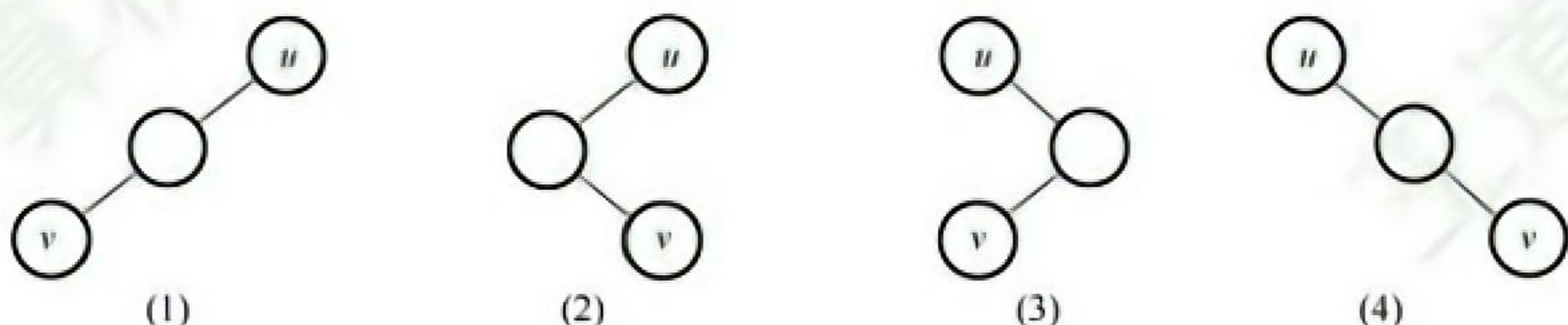


图 III

【另解】由题意可知 u 是 v 的父结点的父结点, 如下图所示, 有四种情况:

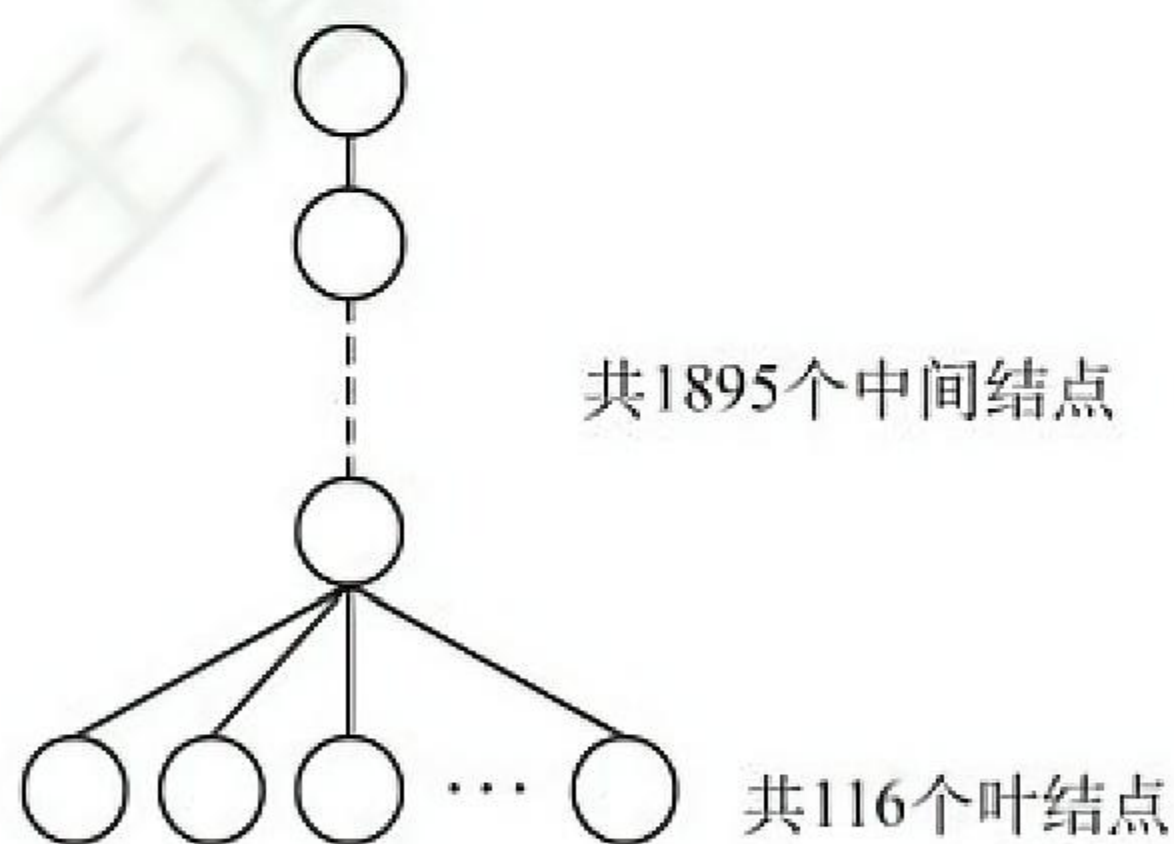


根据树与二叉树的转换规则, 将这四种情况转换成树中结点的关系。(1) 在原来的树中 u 是 v 的父结点的父结点; (2) 在树中 u 是 v 的父结点; (3) 在树中 u 是 v 的父结点的兄弟; (4) 在树中 u 与 v 是兄弟关系。由此可知 I 和 II 正确。

15. D

树转换为二叉树时, 树的每个分支结点的所有子结点中的最右子结点无右孩子, 根结点转换后也没有右孩子, 因此, 对应二叉树中无右孩子的结点个数 = 分支结点数 + 1 = $2011 - 116 + 1 = 1896$ 。

通常本题应采用特殊法求解, 设题意中的树是如下图所示的结构, 则对应的二叉树中仅有前 115 个叶结点有右孩子, 所以无右孩子的结点个数 = $2011 - 115 = 1896$ 。

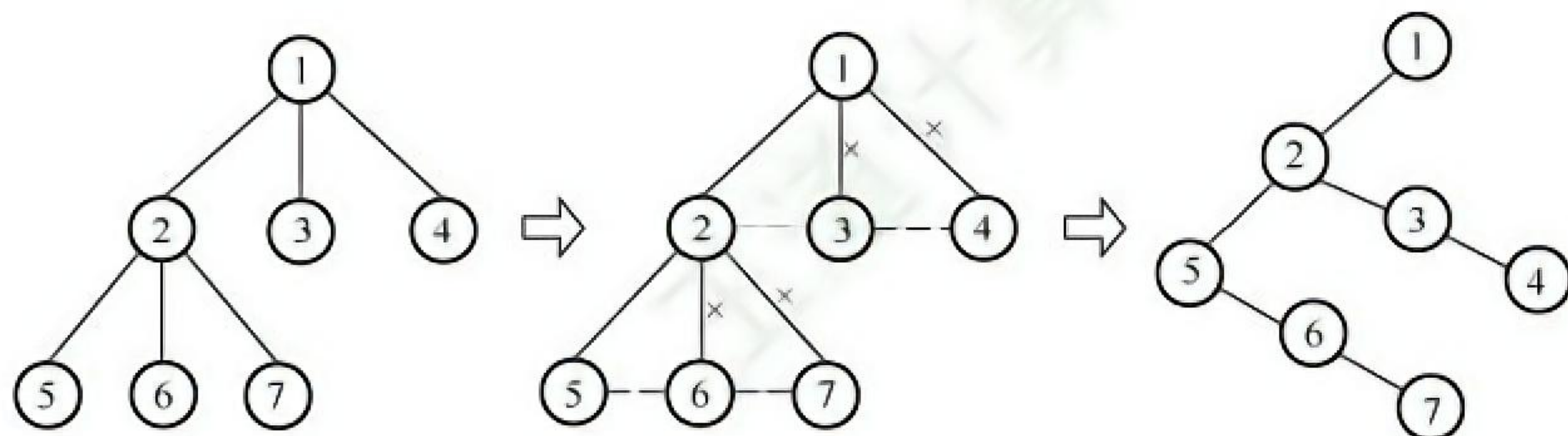


16. C

将森林转化为二叉树相当于用孩子兄弟表示法来表示森林。在变化过程中, 原森林某结点的第一个孩子结点作为它的左子树, 它的兄弟作为它的右子树。森林中的叶结点由于没有孩子结点, 转化为二叉树时, 该结点就没有左结点, 因此 F 中叶结点的个数等于 T 中左孩子指针为空的结点个数。此题还可通过一些特例来排除 A、B 和 D。

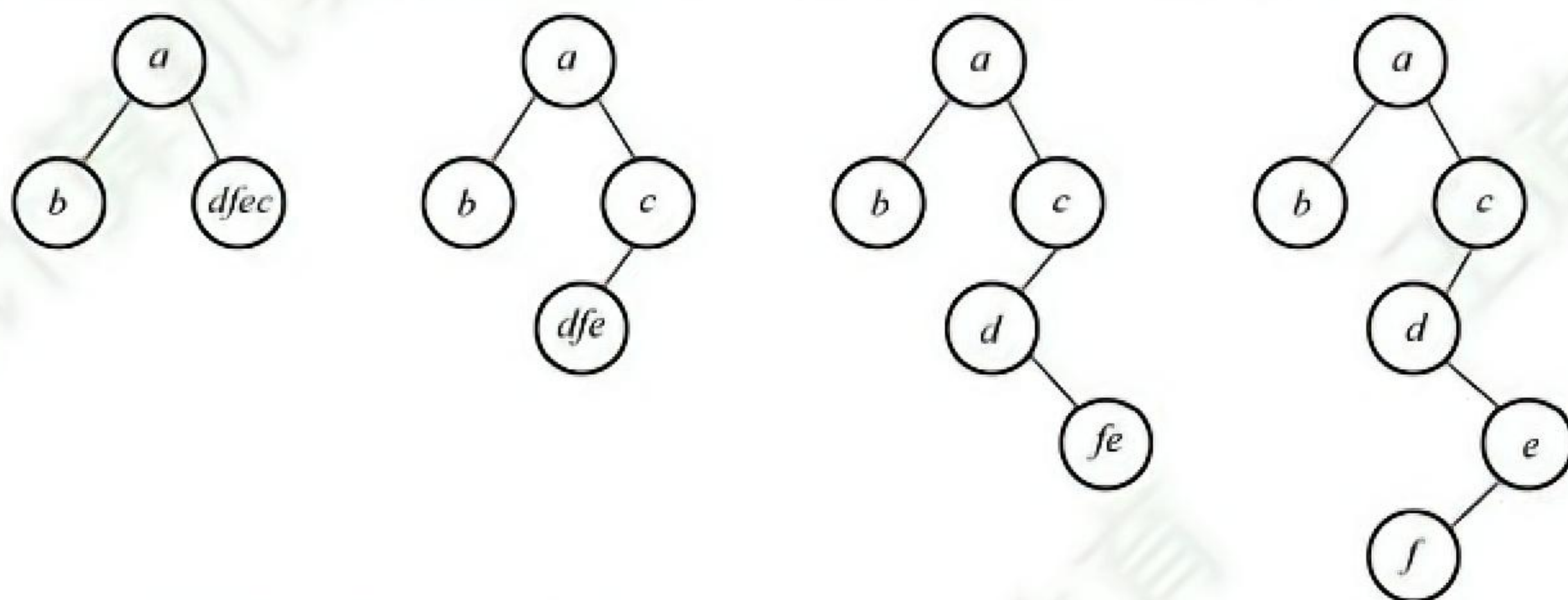
17. B

后根遍历树可分为两步: ①从左到右访问双亲结点的每个孩子 (转化为二叉树后, 先访问根结点, 再访问右子树); ②访问完所有孩子后再访问它们的双亲结点 (转化为二叉树后, 先访问左子树, 再访问根结点), 因此树的后根遍历序列与其相应二叉树的中序遍历序列相同。对于此类题, 采用特殊值法求解通常会更便捷, 左下图树 T 转换为二叉树 BT 的过程如下图所示, 树的后序遍历序列显然和其相应二叉树的中序遍历序列相同, 均为 5,6,7,2,3,4,1。



18. C

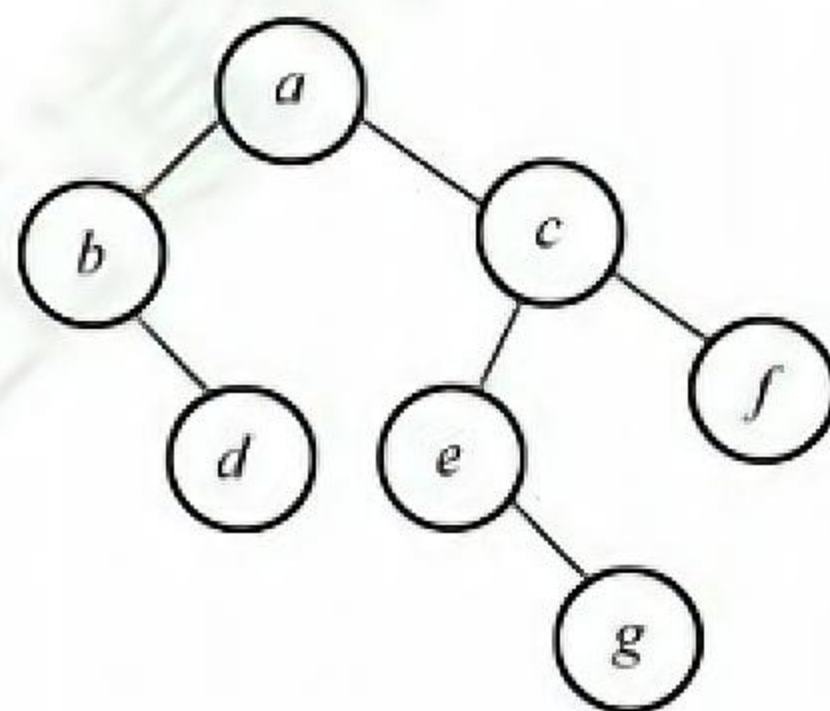
森林 F 的先根遍历序列对应于其二叉树 T 的先序遍历序列, 森林 F 的后根遍历序列对应于其二叉树 T 的中序遍历序列。即 T 的先序遍历序列为 a, b, c, d, e, f , 中序遍历序列为 b, a, d, f, e, c 。根据二叉树 T 的先序序列和中序序列可以唯一确定它的结构, 构造过程如下:



可以得到二叉树 T 的后序序列为 b, f, e, d, c, a 。

19. C

由二叉树 T 的先序序列和中序序列可以构造出 T , 如下图所示。由森林转化成二叉树的规则可知, 森林中每棵树的根结点以右子树的方式相连, 所以 T 中的结点 a, c, f 为 F 中树的根结点, 森林 F 中有 3 棵树。



5.5.4 答案与解析

一、单项选择题

01. A

由哈夫曼树的构造过程可知，哈夫曼树中只有度为 0 和 2 的结点。在非空二叉树中，有 $n_0 = n_2 + 1$ ，所以 $n_2 = n - 1$ 。

【另解】 n 个结点构造哈夫曼树需要 $n - 1$ 次合并过程，每次合并新建一个分支结点，所以选择选项 A。

02. C

首先, 3 和 5 构造为一棵子树, 其根权值为 8, 然后该子树与 6 构造为一棵新子树, 根权值为 14, 再后 9 与 12 构造为一棵子树, 最后两棵子树共同构造为一棵哈夫曼树。

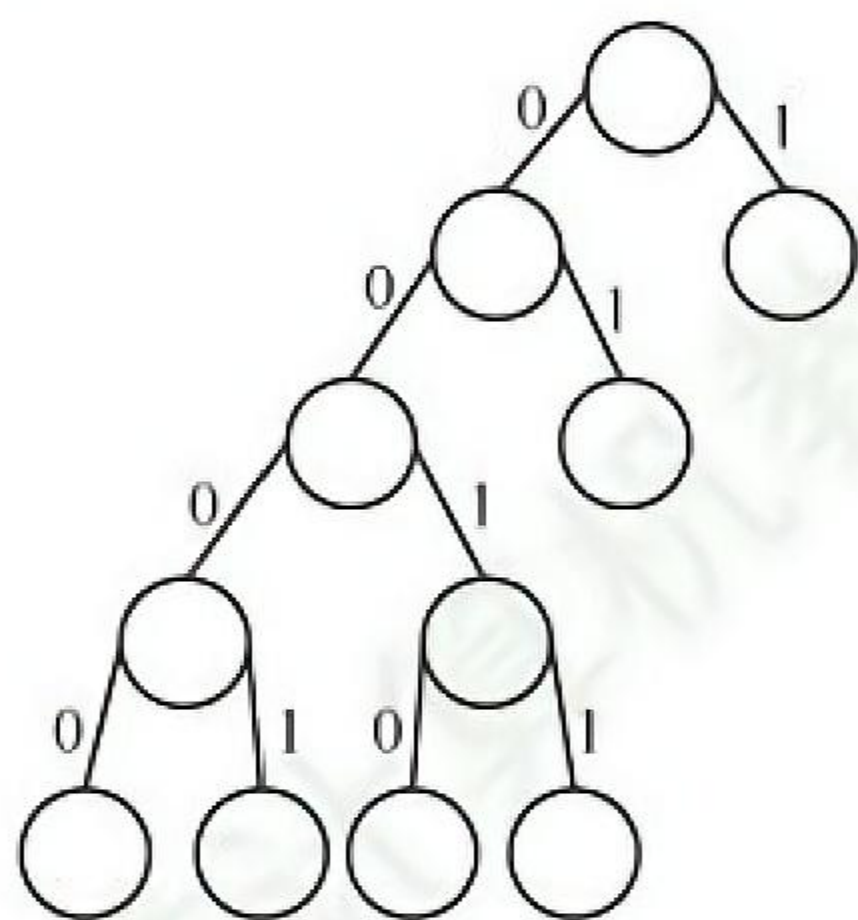
03. B

若没有一个编码是另一个编码的前缀, 则称这样的编码为前缀编码。在选项 B 中, 0 是 00 的前缀, 1 是 11 的前缀。

04. C

在哈夫曼编码中, 一个编码不能是任何其他编码的前缀。3 位编码可能是 001, 对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 000, 对应的 4 位编码只能是 0010 和 0011。若全采用 4 位编码, 则可以为 0000, 0001, 0010 和 0011。题中问的是最多, 所以选择选项 C。

【另解】若哈夫曼编码的长度只允许小于或等于 4, 则哈夫曼树的高度最高是 5, 已知一个字符编码为 1, 另一个字符编码是 01, 这说明第二层和第三层各有一个叶结点, 为使得该树从第 3 层起能够对尽可能多的字符编码, 余下的二叉树应该是满二叉树, 如下图所示, 底层可以有 4 个叶结点, 最多可以再对 4 个字符编码。



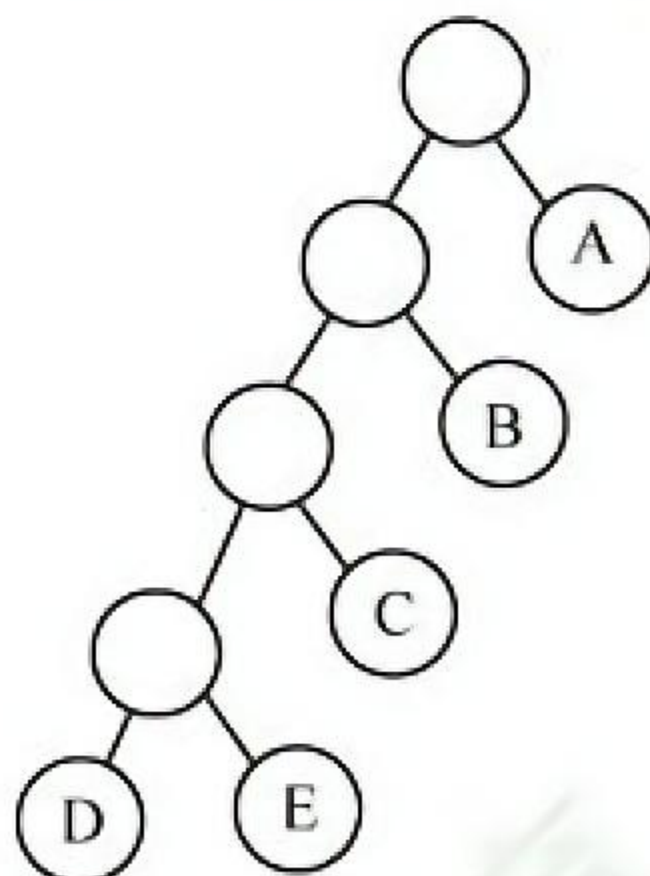
05. B

根据上题的结论, 叶结点数为 $(215 + 1)/2 = 108$, 所以共有 108 个不同的码字。

【另解】在哈夫曼树中只有度为 0 和 2 的结点, 结点总数 $n = n_0 + n_2$, 且 $n_0 = n_2 + 1$, 由题知 $n = 215$, $n_0 = 108$ 。

06. C

在哈夫曼树的构造中, 每个初始结点最终都成为叶结点, 5 个初始结点构造的哈夫曼树共新建 4 个双分支结点, 4 个双分支结点所构成的高度最高的哈夫曼树如下图所示, 其高度是 5。



07. D

哈夫曼树通常是指带权路径长度达到最小的扩充二叉树, 在其构造过程中每次选根的权值最小的两棵树, 一棵作为左子树, 一棵作为右子树, 生成新的二叉树, 新的二叉树根的权值应为其左右两棵子树根结点权值的和。至于谁做左子树, 谁做右子树, 没有限制, 所以构造的哈夫曼树是不唯一

的。哈夫曼树只有度为 0 和 2 的结点，度为 0 的结点是外结点，带有权值，没有度为 1 的结点。

08. C

n 个初始结点构造的哈夫曼树共新建 $n-1$ 个双分支结点，因此哈夫曼树的结点总数是 $2n-1$ ，是个奇数，I 错误。哈夫曼树中没有度为 1 的结点，II 错误。哈夫曼的带权路径长度有两种计算方法：①所有叶结点的带权路径长度之和；②所有分支结点的权值之和，III 正确。

09. C

一棵度为 m 的哈夫曼树应只有度为 0 和 m 的结点，设度为 m 的结点有 n_m 个，度为 0 的结点有 n_0 个，又设结点总数为 N ， $N=n_0+n_m$ 。因有 N 个结点的哈夫曼树有 $N-1$ 条分支，则 $mn_m=N-1=n_m+n_0-1$ ，整理得 $(m-1)n_m=n_0-1$ ， $n_m=(n_0-1)/(m-1)$ 。

10. B

并查集的存储结构是用双亲表示法存储的树，主要是为了方便两个重要的操作。

11. C

初始时，0~9 各自成一个集合。查找 1-2 时，合并{1}和{2}；查找 3-4 时，合并{3}和{4}；查找 5-6 时，合并{5}和{6}；查找 7-8 时，合并{7}和{8}；查找 8-9 时，合并{7, 8}和{9}；查找 1-8 时，合并{1, 2}和{7, 8, 9}；查找 0-5 时，合并{0}和{5, 6}；查找 1-9 时，它们属于同一个集合。最终的集合为{0, 5, 6}、{1, 2, 7, 8, 9}和{3, 4}，因此答案选择选项 C。

12. D

依次探测图的各条边，用并查集检查该边依附的两个顶点是否已属于同一集合（两个顶点的根结点是否相同）。若是，则说明图中存在环路，A 错误。经过路径优化后，并查集在最坏情况下的高度远小于 $O(n)$ ，B 错误。Find 操作总返回当前根结点作为集合的标志，C 错误。

13. D

在用并查集实现 Kruskal 算法求图的最小生成树时：判断是否加入一条边之前，先查找这条边关联的两个顶点是否属于同一个集合（即判断加入这条边之后是否形成回路），若形成回路，则继续判断下一条边；若不形成回路，则将该边和边对应的顶点加入最小生成树 T ，并继续判断下一条边，直到所有顶点都已加入最小生成树 T 。B 正确。用并查集判断无向图连通性的方法：遍历无向图的边，每遍历到一条边，就把这条边连接的两个顶点合并到同一个集合中，处理完所有边后，只要是相互连通的顶点都会被合并到同一个子集合中，相互不连通的顶点一定在不同的子集合中。C 正确。未做路径优化的并查集在最坏情况下的高度为 n ，此时查找操作的时间复杂度为 $O(n)$ ，时间复杂度通常指最坏情况下的时间复杂度。D 错误。

14. A

哈夫曼树为带权路径长度最小的二叉树，不一定是完全二叉树。哈夫曼树中没有度为 1 的结点，选项 B 正确。构造哈夫曼树时，最先选取两个权值最小的结点作为左、右子树构造一棵新的二叉树，选项 C 正确。哈夫曼树中任意一个非叶结点的权值为其左、右子树根结点的权值之和，可知，哈夫曼树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值。

15. D

前缀编码的定义是在一个字符集中，任何一个字符的编码都不是另一个字符编码的前缀。选项 D 中的编码 110 是编码 1100 的前缀，违反了前缀编码的规则，所以选项 D 不是前缀编码。

16. D

在哈夫曼树中，左右孩子权值之和为父结点权值。仅以分析选项 A 为例：若两个 10 分别属于两棵不同的子树，则根的权值不等于其孩子的权值和，不符；若两个 10 属同棵子树，则其权

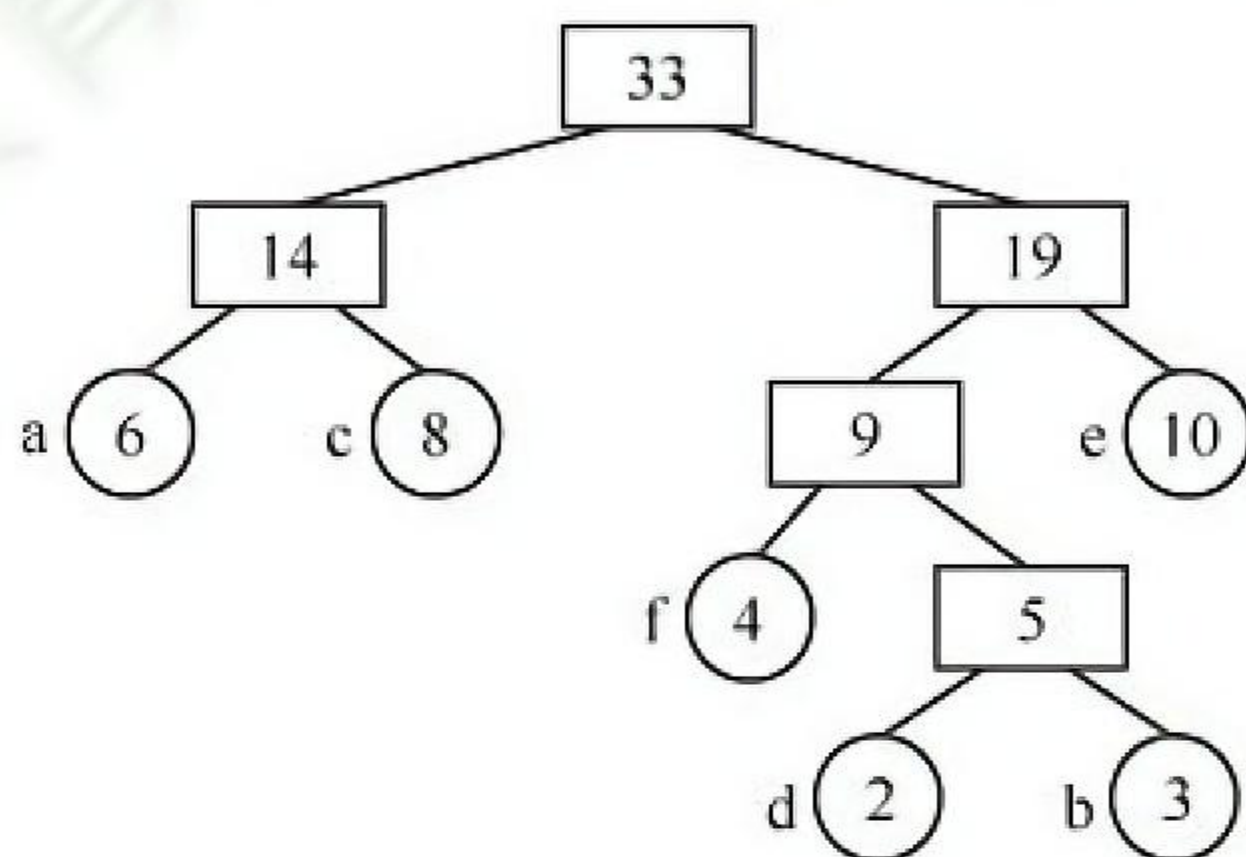
值不等于其两个孩子（叶结点）的权值和，不符。选项 B、C 选项的排除方法相同。

17. D

哈夫曼编码是前缀编码，各个编码的前缀不同，因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101，译码结果是 afeefgd。选项 D 正确。

18. A

根据各字符出现的次数构造的哈夫曼树如下图所示。由图可知，a、c 和 e 的编码长度应该相同；a 和 c 的第 1 个编码应该相同，且与 e 的第 1 个编码不同；b 和 d 的前 3 个编码应该相同。

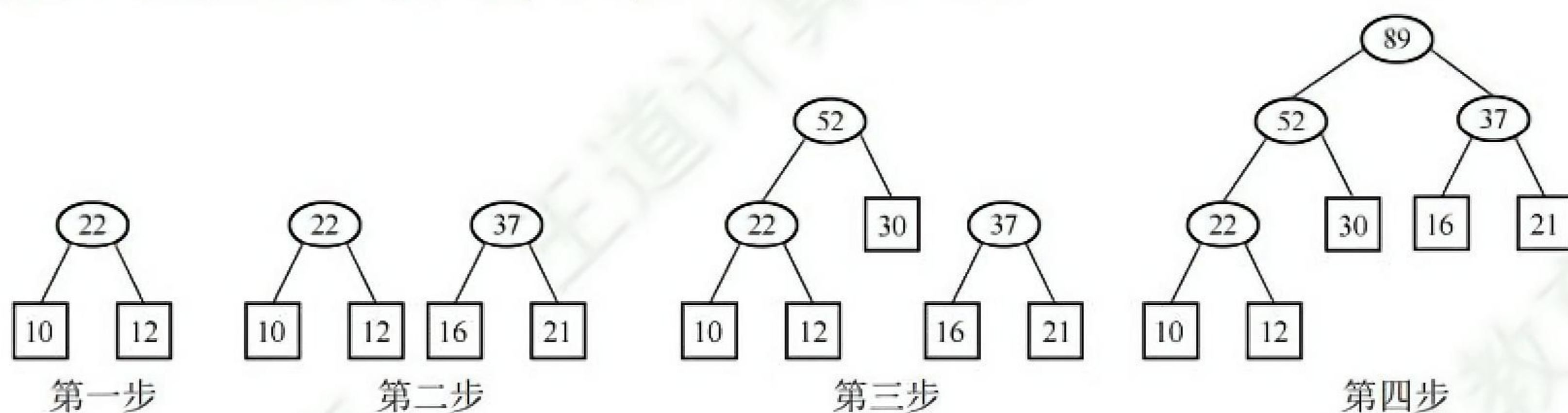


19. C

n 个符号构造哈夫曼树的过程中，共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1=115$ ， n 的值为 58。

20. B

对于带权值的结点，构造出哈夫曼树的带权路径长度（WPL）最小，哈夫曼树的构造过程如下图所示。求得其 $WPL = (10 + 12) \times 3 + (30 + 16 + 21) \times 2 = 200$ 。



21. D

可以画一个简单的特例来证明。图 1 是满足条件的二叉树 T_1 ，图 2 是满足条件的二叉树 T_2 ，结点中有值表示这个结点是编码字符。 T_1 和 T_2 的结点数不同，选项 A 错误。 T_1 的高度等于 T_2 的高度，选项 B 错误。出现频次不同的字符在 T_1 中也可能处于相同的层，选项 C 错误。对于定长编码集，所有字符一定都在 T_2 中处于相同的层，而且都是叶结点。

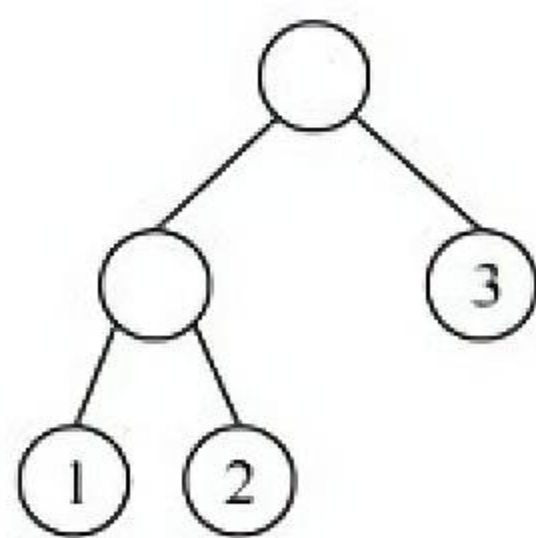


图 1

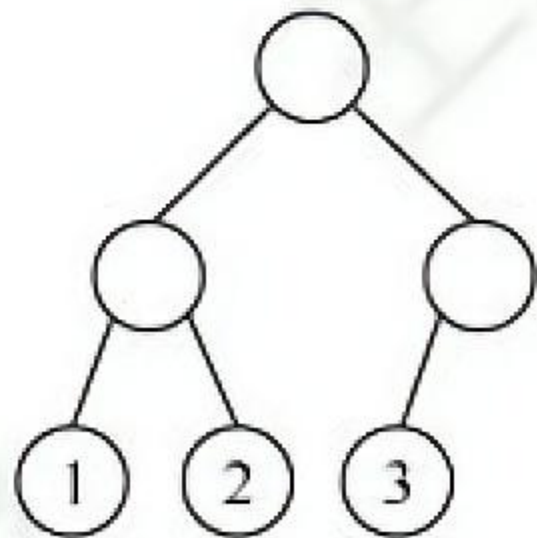
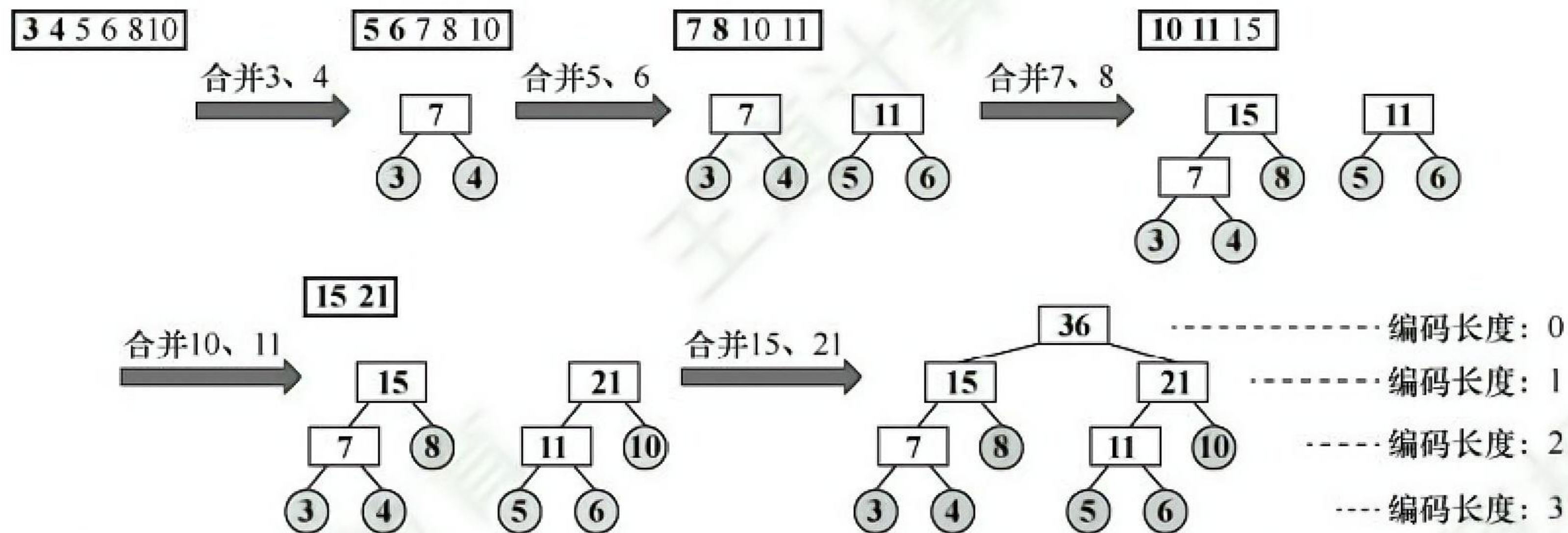


图 2

22. B

构建哈夫曼树的过程如下图所示。



对叶结点的哈夫曼编码, 共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点, 编码的加权平均长度为 $[(3+4+5+6) \times 3 + (8+10) \times 2] / (3+4+5+6+8+10) = 2.5$ 。